

自然语言处理

Natural Language Processing

什么是自然语言处理？

- 自然语言处理（Natural Language Processing, NLP）旨在探索实现人与计算机之间用自然语言进行有效交流的理论与方法。
- 能够理解自然语言的意义
 - 自然语言理解（Natural Language Understanding, NLU）
- 以自然语言文本来表达给定的意图、思想等
 - 自然语言生成（Natural Language Generation, NLG）

背景

- 
- A background image showing three people in a meeting. A man in the center is gesturing with his hands while speaking to two other people, a woman on the left and a man on the right. They are sitting around a table with a laptop and some food. The image has a blue overlay.
- **语言是人类与其他动物最重要的区别**
 - 逻辑思维以语言的形式表达
 - 知识以文字的形式记录和传播
 - **如果人工智能想要获取知识，就必须懂得理解人类使用的不太精确、可能有歧义、混乱的语言。**

自然语言处理的研究历史

- 自然语言处理的研究历史可以追溯到20世纪40年代中后期

- 1947年 Warren Weaver 提出了利用计算机翻译人类语言的可能
- 1949年发布《Translation》（翻译）备忘录
- 1950年 Alan Turing 发表《Computing Machinery and Intelligence》
- 1954年 Georgetown University与IBM 合作的机器翻译演示系统将60多个俄语句子翻译成了英文



VOL. LIX. No. 236.] [October, 1950

MIND
A QUARTERLY REVIEW
OF
PSYCHOLOGY AND PHILOSOPHY

I.—COMPUTING MACHINERY AND
INTELLIGENCE

By A. M. TURING

1. *The Imitation Game.*

I PROPOSE to consider the question, 'Can machines think?' This should begin with definitions of the meaning of the terms 'machine' and 'think'. The definitions might be framed so as to reflect so far as possible the normal use of the words, but this attitude is dangerous. If the meaning of the words 'machine' and 'think' are to be found by examining how they are commonly



自然语言处理的研究历史

● 初创期——20世纪50年代末到60年代

- Noam Chomsky为代表的**符号学派 / 规则学派**提出了形式语言理论，基于1957年发表的《Syntactic Structures》（句法结构）介绍了生成语法的概念，并提出了一种特定的生成语法称为转换语法。开启了使用数学方法研究语言的先河。
- 1959年，Bledsoe和Browning将贝叶斯方法（Bayesian method）应用于字符识别问题，试图通过贝叶斯方法来解决自然语言处理中的问题，开创了**统计学派 / 随机学派**。

在这一阶段，计算语言学（Computational Linguistics）的概念也被正式提出，1962年ACL成立，1965年第一届国际计算语言学大会（The International Conference on Computational Linguistics, COLING）召开。

自然语言处理的研究历史

● 发展期——20世纪70年代到80年代

- 基于规则、逻辑的方法取得了一定成果
- 1970 年，Colmerauer 和他的同事们使用逻辑方法研制了Q 系统（Q-system）和“变形语法”（Metamorphosis grammar），并在机器翻译中得到应用；Colmerauer 还是 Prolog 语言的前驱者，他使用逻辑程序设计的思想设计了 Prolog 语言。

?- likes(john, jane). ← dot necessary

true. ← answer from prolog interpreter

sign on

prolog query

prompt

variables

?- friends(X, Y).

X = john,

Y = jane ; ← type ; to get next solution

X = jane,

Y = john.

自然语言处理的研究历史

● 发展期——20世纪70年代到80年代

- 1972年，麻省理工大学的维诺格拉德（T. Winograd）等人建立了一个用自然语言模拟指挥机器人动作的系统——SHRDLU，该系统使用规则范式模拟了一个玩具积木世界。
- 系统对话的对象是一个具有简单的“手”和“眼”的玩具机器人，它可以操作放在桌子上的具有不同颜色、尺寸和形状的玩具积木，如立方体、棱锥体、盒子等，机器人能够根据操作人员的命令把这些积木捡起来，移动它们去搭成新的积木结构。

SHRDLU: A demonstration proof

Person: *PICK UP A BIG RED BLOCK.*

Computer: OK. (does it)

Person: *GRASP THE PYRAMID.*

Computer: I DON'T UNDERSTAND
WHICH PYRAMID YOU MEAN.

Person: *FIND A BLOCK WHICH IS TALLER THAN THE ONE YOU
ARE HOLDING AND PUT IT INTO THE BOX.*

Computer: BY "IT", I ASSUME YOU MEAN THE BLOCK WHICH IS
TALLER THAN THE ONE I AM HOLDING.

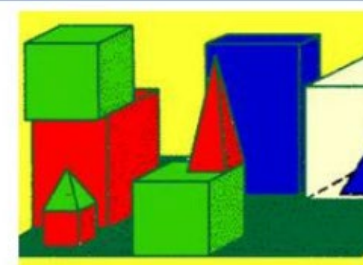
Computer: OK. (does it)

Person: *WHAT DOES THE BOX CONTAIN?*

Computer: THE BLUE PYRAMID AND THE BLUE BLOCK.

Person: *WHAT IS THE PYRAMID SUPPORTED BY?*

Computer: THE BOX.

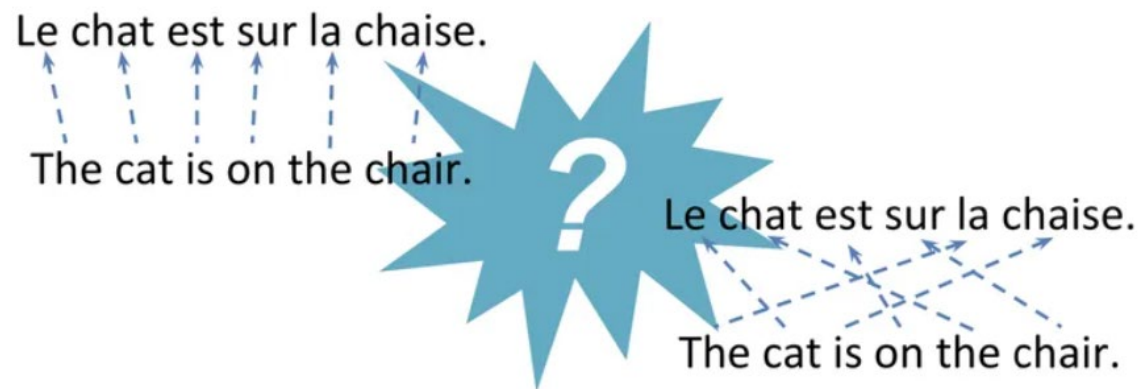


自然语言处理的研究历史

● 反思期——20世纪80年代到90年代

- 计算语言学研究对于过去的研究历史进行了反思，发现过去被否定的有限状态模型和经验主义方法仍然有其合理的内核；
- 20世纪80年代，IBM华生研究中心提出了语音识别概率模型、统计机器翻译思想，并取得了很好的效果；
- “每当我解雇一名语言学家，语音识别器的性能就会提升”（Frederick Jelinek）。

$$\hat{p}_{\text{MLE}}(e \mid \text{Haus}) = \begin{cases} 0.696 & \text{if } e = \text{house} \\ 0.279 & \text{if } e = \text{home} \\ 0.014 & \text{if } e = \text{shell} \\ 0.011 & \text{if } e = \text{household} \\ 0 & \text{otherwise} \end{cases}$$



自然语言处理的研究历史

- 繁荣期——20世纪90年代至今

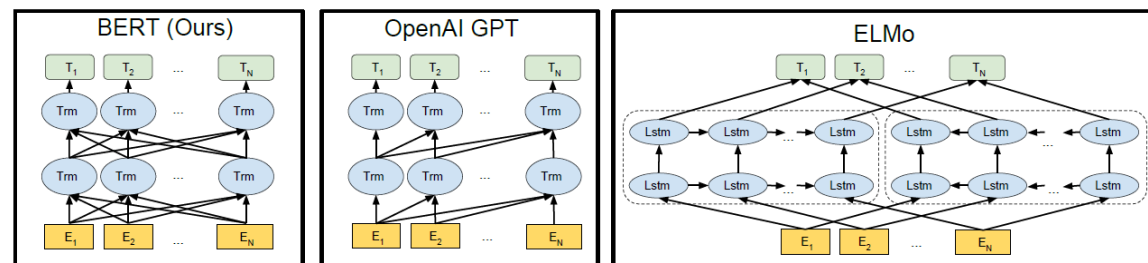
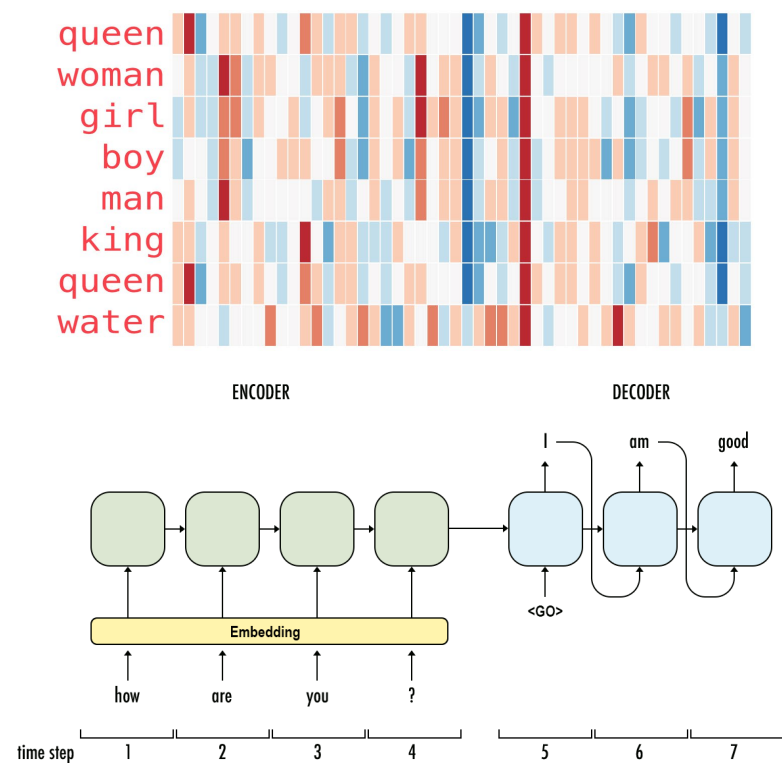
- 2003年，来自亚琛大学（Aachen University）的学者奥赫（F. J. Och）使用统计方法，在很短的时间之内就构造了阿拉伯语和汉语到英语的若干个机器翻译系统。

*Give me enough parallel data, and you can have translation system
for any two languages in a matter of hours.*

自然语言处理的研究历史

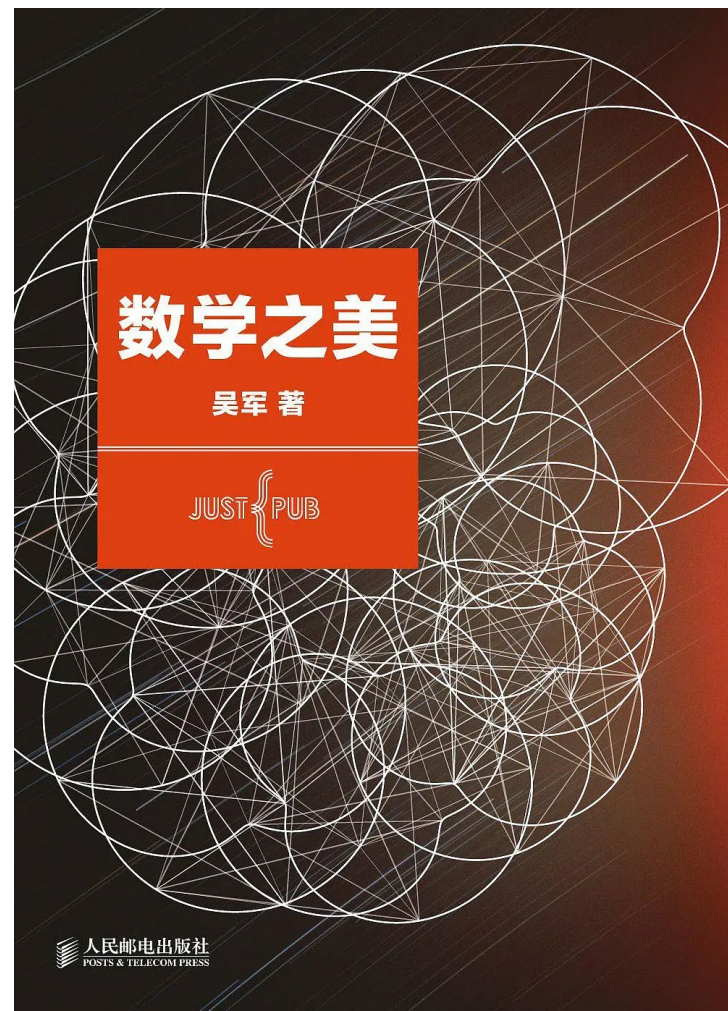
● 繁荣期——20世纪90年代至今

- 概率和数据驱动的方法几乎成为了计算语言学的标准方法；
- 除了传统的机器翻译和信息检索等应用研究进一步得到发展之外，信息抽取、问答系统、自动文摘、术语的自动抽取和标引、文本数据挖掘等新兴的应用研究都有了长足的进展；
- 互联网的出现和“信息爆炸”为自然语言处理研究提供了海量数据；
- Word2Vec、Seq2Seq；
- ELMo、BERT、GPT。

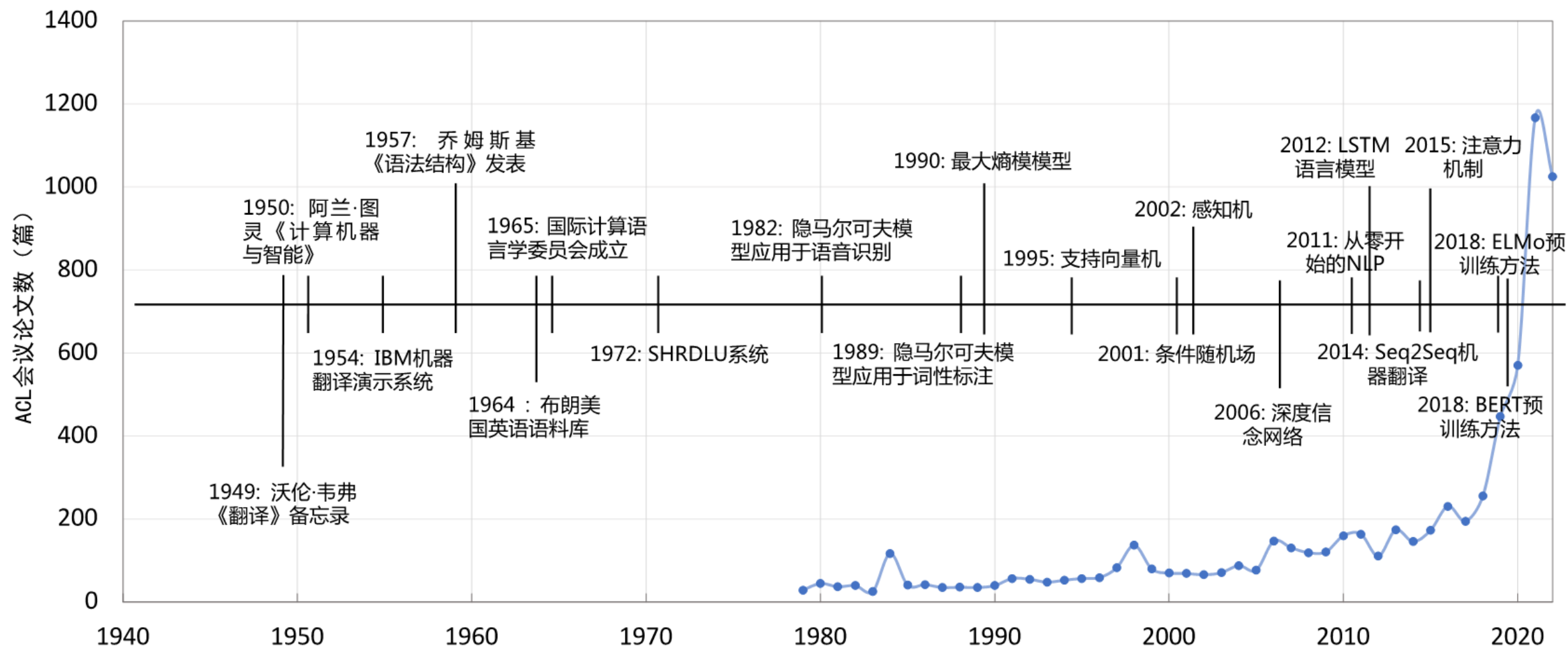


自然语言处理的研究历史

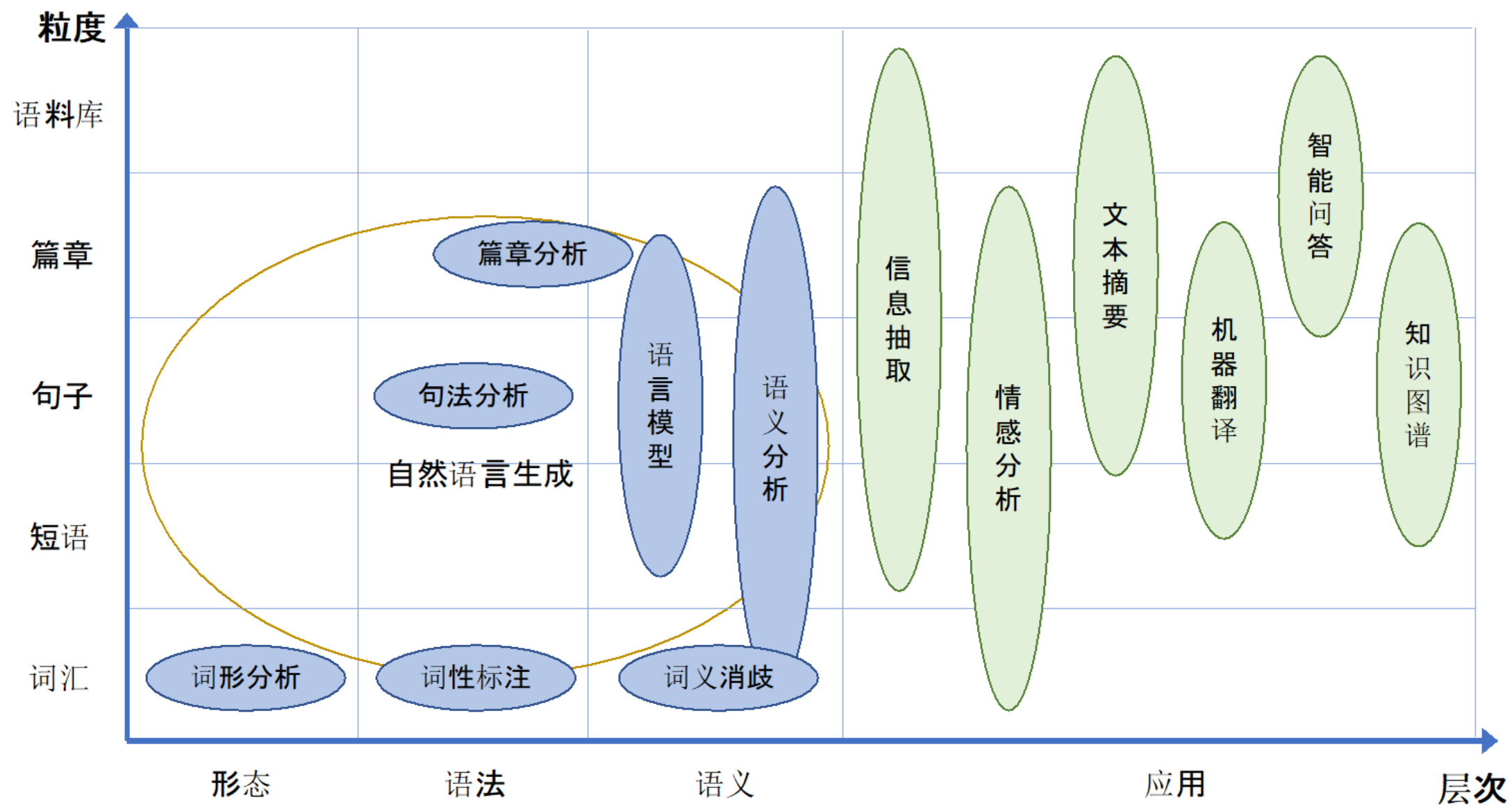
- 繁荣期——20世纪90年代至今



自然语言处理的研究历史



自然语言处理的研究内容



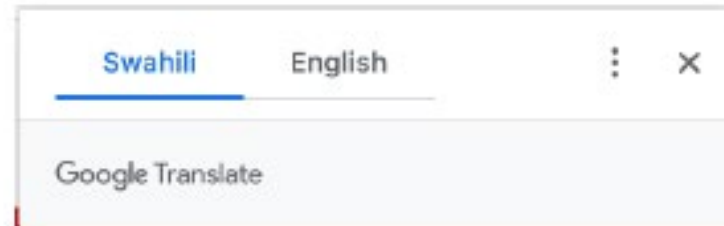
自然语言处理的研究内容

- 机器翻译是自然语言处理领域的早期成功实践。



Malawi yawapoteza mawaziri 2 kutokana na maafa ya COVID-19

TUKO.co.ke imefahamishwa kuwa waziri wa serikali ya mitaa Lingson Belekanyama na mwenzake wa uchukuzi Sidik Mia walifariki dunia ndani ya saa mbili tofauti.



Malawi loses 2 ministers due to COVID-19 disaster

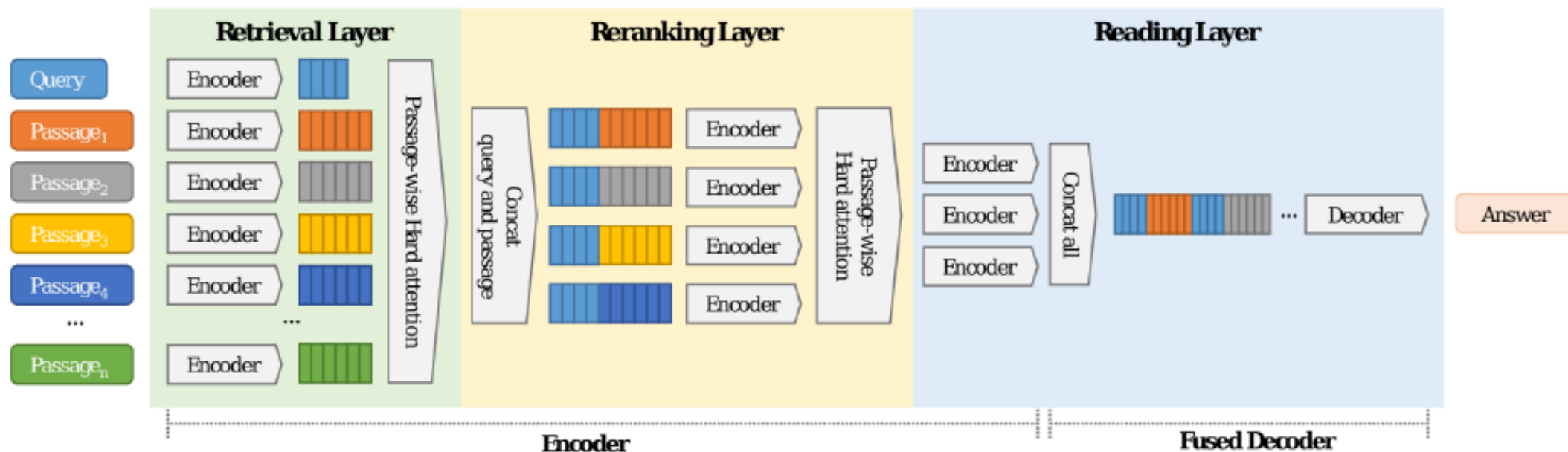
TUKO.co.ke has been informed that local government minister Lingson Belekanyama and his transport counterpart Sidik Mia died within two separate hours.

- “下一代的搜索引擎”

when did Kendrick lamar's
first album come out?

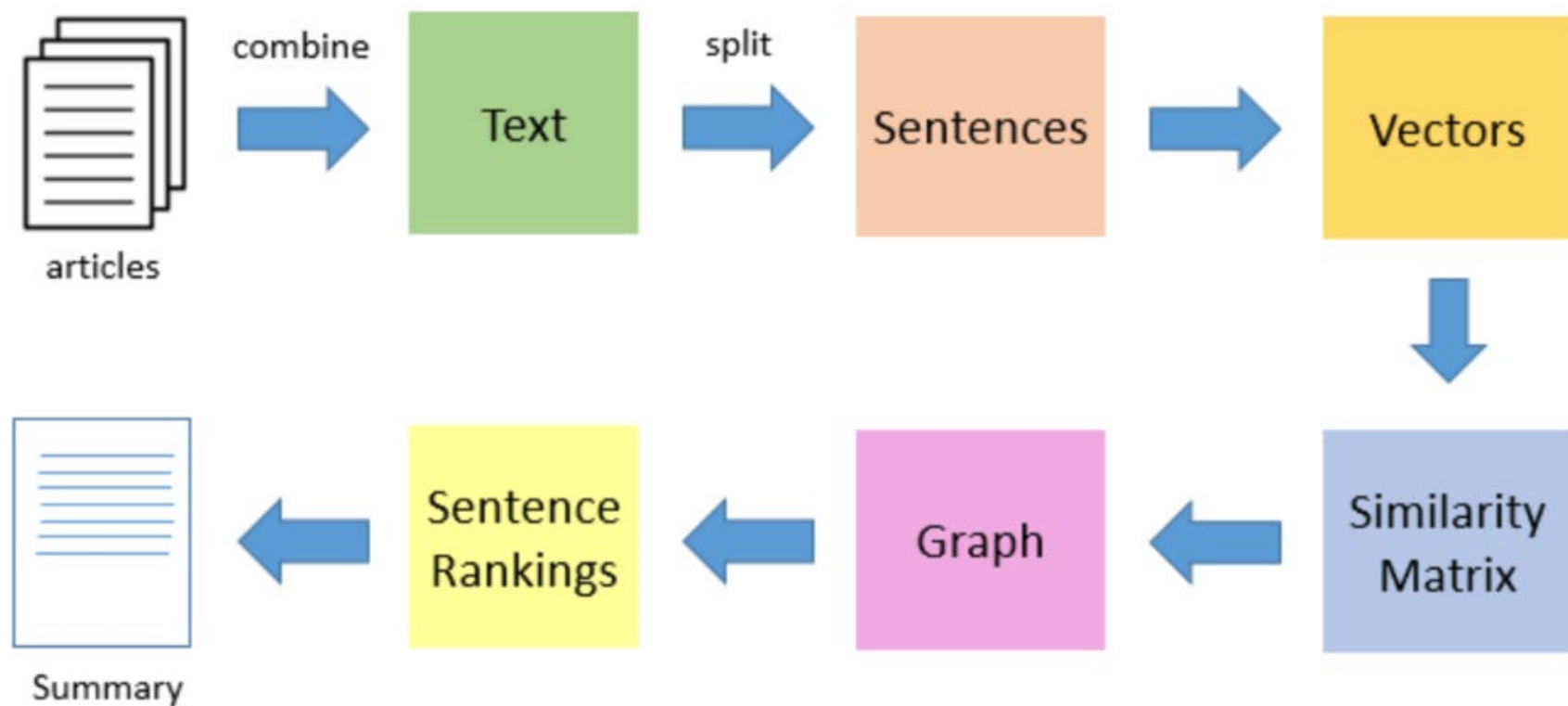
July 2, 2011

These are my leftover songs you all can have them. I'm going to put my best out. My best effort. I'm trying to look for an album in 2012."^[44] In June 2011, Lamar released "Ronald Reagan Era (His Evils)", a cut from *Section.80*, featuring Wu-Tang Clan leader RZA.^[45] On July 2, 2011, Lamar released *Section.80*, his first independent album. The album features guest appearances from GLC, Colin Munroe, Schoolboy Q, and Ab-Soul, while the production was handled by Top Dawg in-house



文本信息摘要

- 信息爆炸时代，总结摘要非常重要
- 将长文本压缩为精简的摘要，保留核心内容



语音识别

- 人机交互的重要入口
- 手机智能助理、智能家居、智能驾驶.....





A illustration from a graphic novel. A bustling city street under the shine of a full moon. The sidewalks bustling with pedestrians enjoying the nightlife. At the corner stall, a young woman with fiery red hair, dressed in a signature velvet cloak, is haggling with the grumpy old vendor. the grumpy vendor, a tall, sophisticated man is wearing a sharp suit, sports a noteworthy moustache is animatedly conversing on his steampunk telephone.

● 自然语言处理的通用基础模型

A

Hey please draft a polite mail to explain my boss Jeremy that I would not be able to come to office for next 2 days because my 9 year son Peter is angry with me that I am not giving him much time. So I have plans to take him out for a trip. I will return and complete my work by meeting deadline.



Dear Jeremy,



I hope this email finds you well. I'm writing to let you know that I will not be able to come into the office for the next two days.

My 9-year-old son Peter is feeling upset with me because he feels like I haven't been giving him enough attention lately. In order to make it up to him, I've decided to take him on a little trip to help us reconnect.

I understand that this may cause some inconvenience, but I promise to make up for any lost time by working extra hard to meet any deadlines when I return.

Thank you for your understanding.

Best regards,
[Your Name]

User What is unusual about this image?



Source: [Barnorama](#)

GPT-4 The unusual thing about this image is that a man is ironing clothes on an ironing board attached to the roof of a moving taxi.

自然语言处理研究的主要难点

自然语言处理研究的主要难点

- **自然语言处理难的根本原因：自然语言在各个层面都广泛存在的各种各样的歧义性**
 - 自然语言文本从形式上是由字符（包括中文汉字、英文字母、符号）组成的字符串
 - 由字母或者汉字可以组成词，由词可以组成词组，由词组可以组成句子，进而组成段落、篇章
 - 无论哪种粒度的语言单元，还是从一个层级向上一个层级转变中都存在歧义和多义现象
- **形式上一样的字符串，可以理解为不同的词串、词组串，并有不同的意义**
- **Joseph F. Kess 和 Ronald A. Hoppe 甚至还提出了“语言无处不歧义”理论**

● 语音歧义 (Phonetic Ambiguity)

- 主要体现在口语中，是由于语言中同音异义词 (Homophone)、爆破音不完全、重音位置不明确等原因造成的；
- 汉语（普通话）中只有413个不同的音（节），如果结合声调的变化组合，也仅有1277个音（节），但是汉字数量则多达数万个，多字同音现象广泛存在；
- 英语中连读、爆破音、重音位置等造成的语音异义也非常常见。

例如： 请问您贵姓？ 免贵姓 Zhang （张？ 章？ ）

chéng shì: 城市、程式、成事、乘势

Please hand me the flower. 请把花递给我。

Please hand me the flour. 请把面粉递给我。

- 语音歧义 (Phonetic Ambiguity)

施氏食狮史

赵元任

石室诗士施氏，嗜狮，誓食十狮。施氏时时适市视狮。十时，适十狮适市。是时，适施氏适市。施氏视是十狮，恃矢势，使是十狮逝世。氏拾是十狮尸，适石室。石室湿，氏使侍拭石室。石室拭，施氏始试食是十狮尸。食时，始识是十狮尸，实十石狮尸。试释是事。

词语切分歧义

• 词语切分歧义 (Word Segmentation Ambiguity)

- 由字符组成词语时的歧义现象
- 英语等印欧语系的语言来说，绝大部分单词之间都由空格或标点分割
- 汉语、日语等语言来说，单词之间通常没有分隔符

例如：语言学是一门基础学科。
这门语言学起来很困难。
你认为学生会听老师的吗？
南京市长江大桥



- 词义歧义 (Word Sense Ambiguity)

- 指同一个词语在不同语境下有多种不同的意义

例如：“打”字在《现代汉语词典（第七版）》中，有两个读音“dá”和“dǎ”，分别作为量词、动词和介词，在作为动词时“打”字有 24 个意项

打 dǎ 动词：

- (1) 用手或器具撞击物体：~ 门 | ~ 鼓
- (2) 器皿、蛋类等因撞击而破碎：碗 ~ 了 | 鸡飞蛋 ~
- (3) 殴打；攻打：~ 架 | ~ 援
- (4) 发生与人交涉的行为：~ 官司 | ~ 交道
- (5) 汲取；盛取：~ 米 | ~ 酱油

词义歧义

● “语言通货膨胀”：

- 定义：一个熟知的表达或符号，因长期频繁使用，其所代表的含义、表达力度或表达效果丢失或改变了，即曾经的语言表达或符号如同货币般贬值了。

😏 这不是笑，这是骂人，千万不要用
😏 这是笑但很油腻，男老师不要对女学生用
😏 这个确实是笑但比较娘，男老师不要用
😏 这个笑很傻逼，同时有些幸灾乐祸
😏 这是笑但是有点憨
😏 这是笑但是有点坏，甚至有点幸灾乐祸
😏 这个是哇哦并且有星星眼
😏 这根本不是笑，这是纠结，很难办的

😏 这是意思就有点多了，原意是破涕为笑，但是大家都觉得是笑哭了，现在又延伸出来小尴尬的意思
😏 这个是捂脸，又哭又笑，哭笑不得
😏 这是不怀好意的笑，但又带点友好，很难解释，您自己体会
😏 这是开心的笑，并且比了个耶
😏 这表示OK欧克没问题，交给我吧
666 这个重点不在笑，而在于点赞，老铁双击666

呵呵 这是骂人，千万不要用
呵呵呵 还是骂人
呵呵呵呵呵 骂人到极致
哈哈 这是敷衍
哈哈 还是敷衍
哈哈哈哈哈（至少五个哈）这才是笑

😏 这也是嘲笑，但程度较低
😏 这个千万不要用，这不是再见，这是让你滚开
👉 这个确实是拜谢对方，但年轻学生会觉得你高高在上
👉 这也是一种阴阳怪气，不要以为这是点赞
OK 这个非常高高在上
OKK 这才是OK

一组老照片，20岁看不懂，30岁看完沉默，40岁看完流泪！



震惊！看完CS: GO中国元年回顾后，CS玩家都沉默了



订阅号 职场创业 阅读

周润发刘德华陈宝国抱过的这三个小孩，如今竟然都成了明星！



订阅号 八卦了啦 阅读 210

结构歧义

- **结构歧义 (Structural Ambiguity)**

- 指由词组成词组或者句子时，由于其组成的词或词组间可能存在不同的语法或语义关系而出现的（潜在）歧义现象。

- “VP+ 的 + 是 +NP” 型歧义结构

例如：反对 | 的 | 是 | 少数人

- “VP+N1+ 的 +N2” 型歧义结构

例如：咬死了 | 猎人 | 的 | 狗

- “N1+ 和 +N2+ 的 +N3” 型歧义结构

例如：桌子 | 和 | 椅子 | 的 | 腿

指代和省略歧义

- **指代歧义 (Demonstrative Ambiguity) :**

- 代词（如我，你，他等）和代词词组（如“那件事”，“这一点”等）所指的事件可能存在歧义

例如：

搜集史料不容易，鉴定和运用史料更不容易，中国过去大部分史学家主要力量就在这方面。

我来到这个厂不久，就发现王主任对自己的办公环境很关心。

- **省略歧义 (Ellipsis Ambiguity) :**

- 自然语言中由于省略所产生的歧义

例如：

我看见张原扶着一位老人走下车来，手上提着一个黑色皮包。

语用歧义

- **语用歧义 (Pragmatic Ambiguity) :**

- 由于上下文、说话人属性、场景等语用方面的原因造成的歧义
- 一句话在不同的场合、由不同的人说、不同的语境，都可能产生不同的理解

例如：

女子致电男友：地铁站见。如果你到了我还没到，你就等着吧。如果我到了你还没到，你就等着吧！！

- 自然语言并不是一成不变的，而是在动态发展中，存在大量未知语言现象，新词汇、新含义、新用法、新句型等层出不穷

例如：新词汇：双碳、双减、绝绝子、社恐、元宇宙

新含义：躺平、打工人、凡尔赛、青蛙、潜水、盖楼

新用法：走召弓虽、YYDS、回忆杀、求扩列、orz

自然语言处理研究方法

自然语言处理的基本范式

- **发展阶段与范式：**

- 自然语言处理的发展经历了从理性主义到经验主义，再到深度学习三大历史阶段
- 基于规则的方法、基于机器学习的方法以及基于深度学习的方法三种范式
- 三种范式基本对应了自然语言处理的不同发展阶段的重点

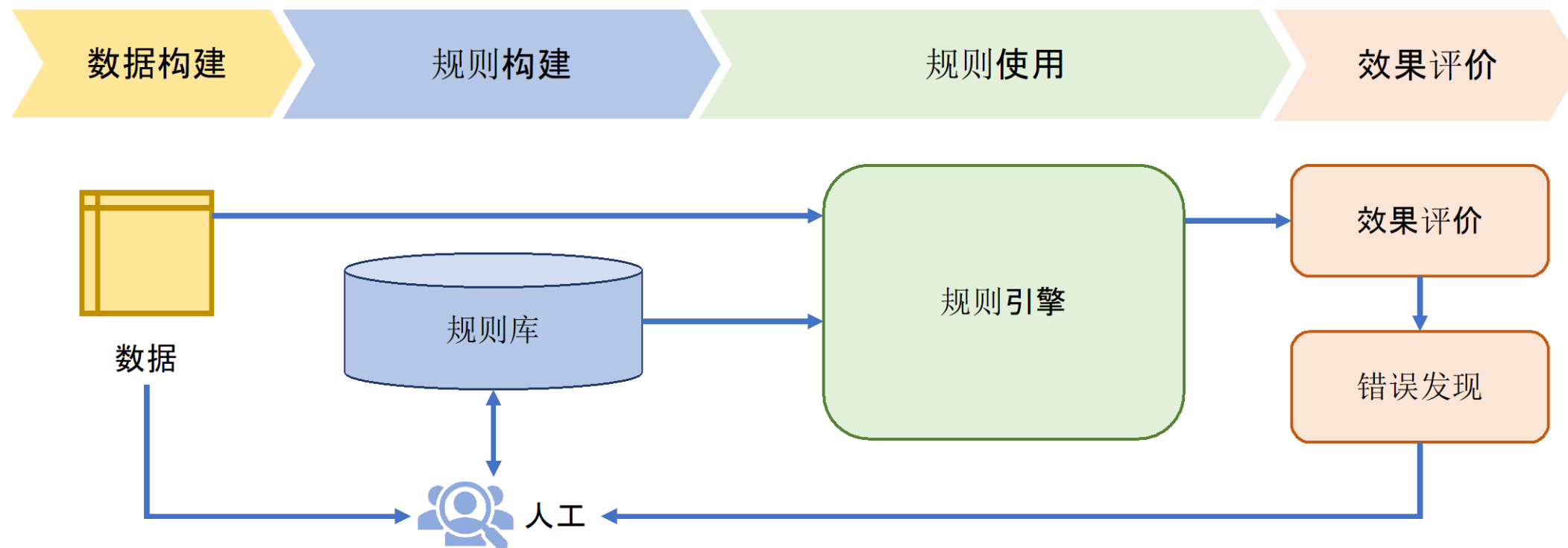
- **需要特别说明的是：**

- 虽然以上三种范式来源于自然语言处理的不同发展阶段，有明显的发展先后顺序，而且在大部分自然语言处理任务的标准评测集合中，基于深度学习的方法都好于基于规则的方法，但是，这三种范式各有利弊，各自都有适合的场景，在实际应用中需要根据任务的特点、计算量、可控制性以及可解释性等具体情况进行选择。

基于规则的方法

● 基于规则的方法：

- 通过词汇、形式文法等制定的规则引入语言学知识，从而完成相应的自然语言处理任务



基于规则的方法

● 基于规则的方法：

- 对于机器翻译任务可以构造如下规则库：
 - IF 源语言主语 = 我 THEN 英语译文主语 = I
 - IF 英语译文主语 = I THEN 英语译文 be 动词为 am/was
 - IF 源语言 = 苹果 AND 没有修饰量词 THEN 英语译文 = apples
- 基于规则的方法从某种程度上可以说，是在试图模拟人类完成某个任务时的思维过程

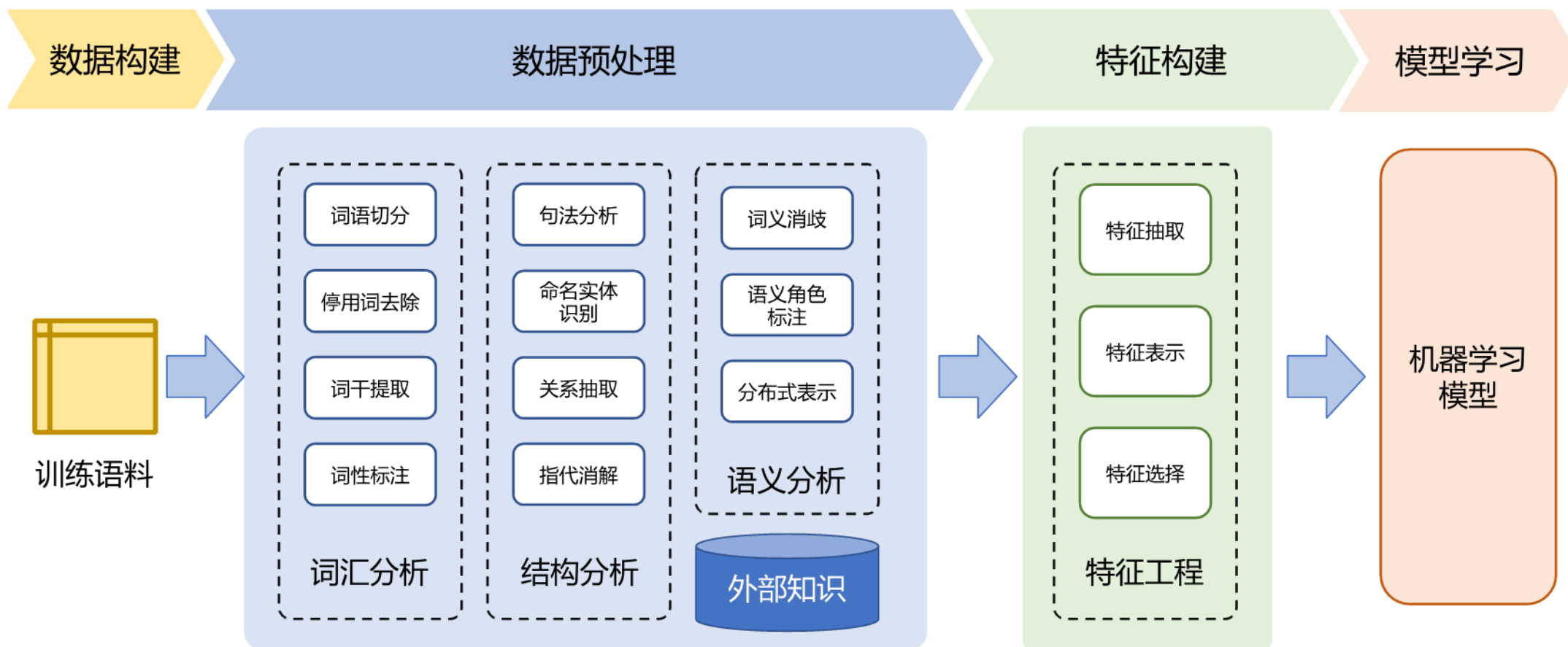
● 优缺点：

- 优点：直观、可解释、不依赖大规模数据；利用规则所表达出来的语言知识具有一定的可读性，不同的人之间可以相互理解
- 缺点：覆盖率差、大规模规则构建代价大、难度高等

基于机器学习的方法

● 基于机器学习的方法：

- 将自然语言处理任务转化为某种分类任务



基于机器学习的方法

- 分为四个步骤：数据构建、数据预处理、特征构建、模型学习

- 数据构建：主要工作是针对任务的要求构建训练语料，也称为**语料库 (Corpus)**
- 数据预处理：主要工作是利用自然语言处理基础算法对原始输入，从词汇、句法、结构、语义等层面进行处理，为特征构建提供基础
- 特征构建：主要工作是针对不同任务从原始输入、词性标注、句法分析、语义分析等结果和数据中提取对于机器学习模型有用的**特征**
- 模型学习：主要工作是根据任务，选择合适的机器学习模型，确定学习准则，采用相应的优化算法，利用语料库**训练模型参数**

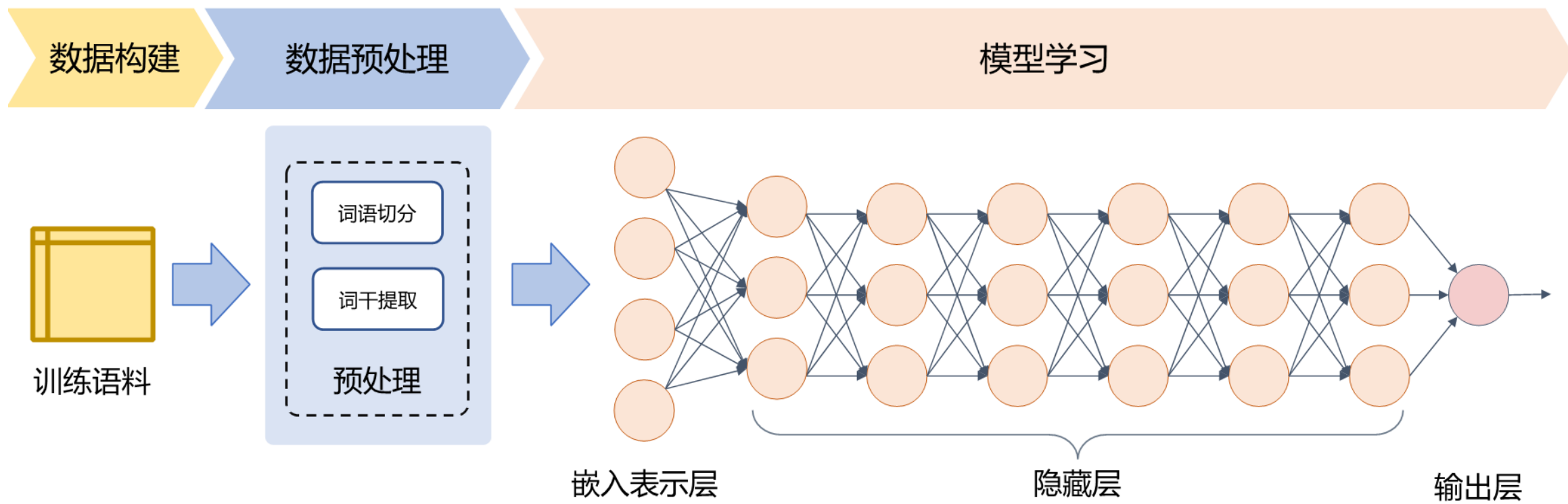
- 特点：

- 以人工特征构建为核心，需要选择合适的机器学习模型
- 效果通常比基于规则的方法好
- 但需要人工参与和选择的环节多，依赖经验

基于深度学习的方法

● 基于深度学习的方法：

- 将特征学习和预测模型融合，通过优化算法使得模型自动地学习出好的特征表示，并基于此进行结果预测



基于深度学习的方法

- 分为三部分：数据构建、数据预处理、模型学习

- 在数据预处理方面大幅度简化，仅包含非常少量的模块
- 通过多层的特征转换，将原始数据转换为更抽象的表示。这些学习到的表示可以在一定程度上完全代替人工设计的特征，这个过程也叫做**表示学习 (Representation Learning)**
- 利用自监督任务对模型进行预训练，通过海量的语料学习到更为通用的语言表示，然后根据下游任务对预训练网络进行调整

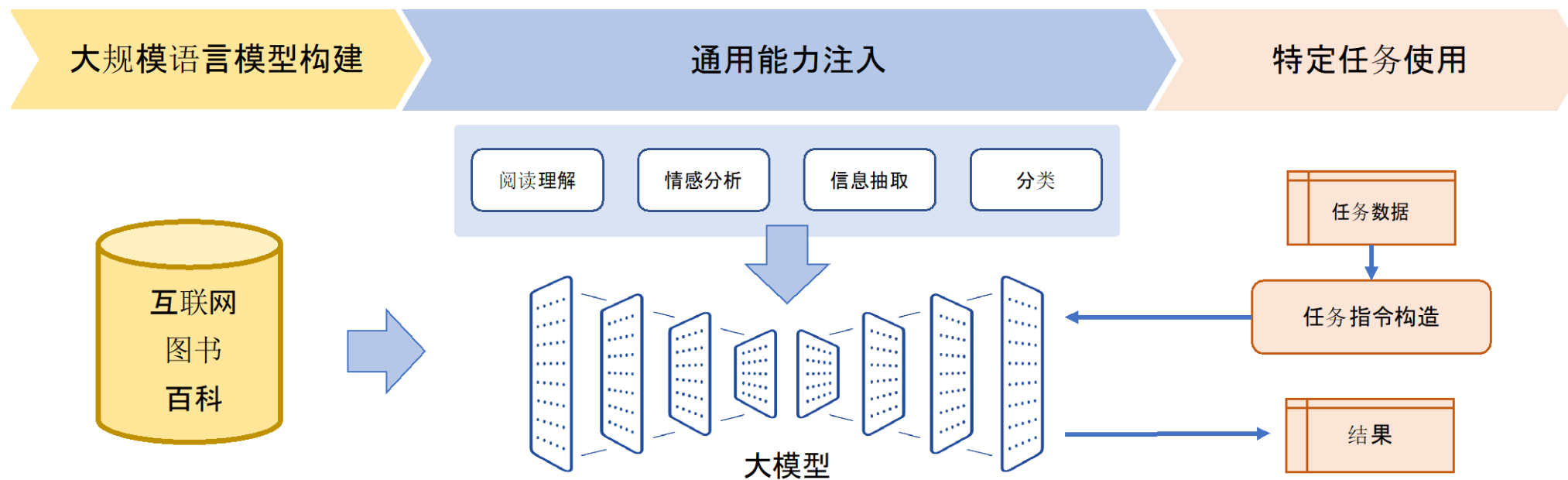
- 特点：

- 人工干预少，在多种自然语言处理任务中都取得了极佳的效果
- 但数据、算力消耗很大

基于深度学习的方法

● 大语言模型：

- 将大量各类型自然语言处理任务，统一为生成式自然语言理解框架
- 在大规模语言模型构建阶段，通过大量的文本内容，训练模型长文本的建模能力，使得模型具有语言生成能力，并使得模型获得隐式的世界知识



词向量

如何表示字词的含义？

● 早期探索

- WordNet: 一个大型英语词汇数据库，将单词之间的关系用概念网络表示，单词之间的主要关系是同义词（Synonym）和上位词（Hypernym）。

PRINCETON UNIVERSITY

WordNet

A Lexical Database for English

What is WordNet

People

News

Use Wordnet Online

Download

Citing WordNet

License and Commercial Use

Related Projects

Documentation

Publications

Frequently Asked Questions

What is WordNet?

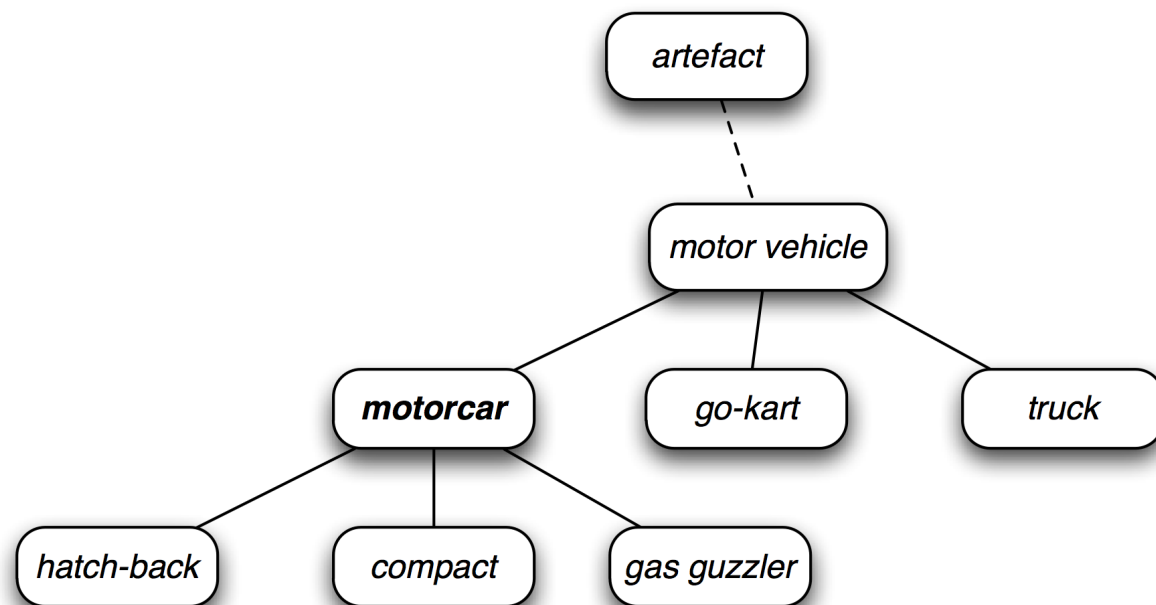
Any opinions, findings, and conclusions or recommendations expressed in this material are those of the creators of WordNet and do not necessarily reflect the views of any funding agency or Princeton University.

When writing a paper or producing a software application, tool, or interface based on WordNet, it is necessary to properly **cite the source**. Citation figures are critical to WordNet funding.

About WordNet

WordNet® is a large lexical database of English. Nouns, verbs, adjectives and adverbs are grouped into sets of cognitive synonyms (synsets), each expressing a distinct concept. Synsets are interlinked by means of conceptual-semantic and lexical relations. The resulting network of meaningfully related words and concepts can be navigated with the **browser**. WordNet is also freely and publicly available for **download**. WordNet's structure makes it a useful tool for computational linguistics and natural language processing.

WordNet superficially resembles a thesaurus, in that it groups words together based on their meanings. However, there are some important distinctions. First, WordNet interlinks not just word forms—strings of letters—but specific senses of words. As a result, words that are found in close proximity to one another in the network are semantically disambiguated. Second, WordNet labels the semantic relations among words, whereas the groupings of words in a thesaurus does not follow any explicit pattern other than meaning similarity.



如何表示字词的含义？

● 早期探索

e.g., synonym sets containing “good”:

```
from nltk.corpus import wordnet as wn
poses = { 'n': 'noun', 'v': 'verb', 's': 'adj (s)', 'a': 'adj', 'r': 'adv' }
for synset in wn.synsets("good"):
    print("{}: {}".format(poses[synset.pos()],
        ", ".join([l.name() for l in synset.lemmas()])))
```

```
noun: good
noun: good, goodness
noun: good, goodness
noun: commodity, trade_good, good
adj: good
adj (sat): full, good
adj: good
adj (sat): estimable, good, honorable, respectable
adj (sat): beneficial, good
adj (sat): good
adj (sat): good, just, upright
...
adverb: well, good
adverb: thoroughly, soundly, good
```

e.g., hypernyms of “panda”:

```
from nltk.corpus import wordnet as wn
panda = wn.synset("panda.n.01")
hyper = lambda s: s.hypernyms()
list(panda.closure(hyper))
```

```
[Synset('procyonid.n.01'),
Synset('carnivore.n.01'),
Synset('placental.n.01'),
Synset('mammal.n.01'),
Synset('vertebrate.n.01'),
Synset('chordate.n.01'),
Synset('animal.n.01'),
Synset('organism.n.01'),
Synset('living_thing.n.01'),
Synset('whole.n.02'),
Synset('object.n.01'),
Synset('physical_entity.n.01'),
Synset('entity.n.01')]
```

如何表示字词的含义？

● WordNet的主要问题

- 没有考虑语境和上下文（而语境、上下文在自然语言处理中非常重要）；
- 缺少最新词汇，难以做到实时更新（语言是动态的、不断发展的，每年都会出现很多新词汇）；
- 需要耗费人力进行创建、维护，工作量很大；
- WordNet的构建是主观的，而不是基于客观数据的；
- 较粗糙，难以精确地衡量词汇之间的相似度。

将字词表示为向量

- 离散表示 (One-hot表示)

- 将每个词表示为One-hot向量, 例如:

`motel = [0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]`

`hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0]`

- 向量的维度=词典中的词汇数量 (50万以上)
- 主要问题:
 - 词汇数量很多, 向量维度太高;
 - 难以计算词汇之间的相似性 (每两个向量都是正交的) 。

将字词表示为向量

• 分布式表示（连续表示）

- 现代自然语言处理领域最成功的技术思想之一；
- 字词的含义与跟它经常一起出现的词密切相关（J. R. Firth）；
- 例如：
 - 当一个单词 w 在文本中出现时，其上下文（Context）就是与之一一起出现的单词（在某个固定长度的窗口）；
 - 我们可以使用很多 w 的上下文构建 w 的向量表示；
 - 如在下面的上下文中都出现了 banking 这个词：



*You shall know a word by the
company it keeps.*
——J. R. Firth, 1957

*...government debt problems turning into **banking** crises as happened in 2009...*
*...saying that Europe needs unified **banking** regulation to replace the hodgepodge...*
*...India has just given its **banking** system a shot in the arm...*

将字词表示为向量

● 词向量

- 为每个单词构建一个密集向量 (Dense vector) —— 向量的每个元素都是一个浮点数;
- 在相似上下文中出现的单词, 词向量也是“相似”的;
- 可以用向量点积计算词与词之间的相似度;
- 词向量 (Word vector) 也称为词嵌入 (Word embedding) 或者词的表示 (Word representation) 。

$$\textit{banking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix} \qquad \textit{monetary} = \begin{pmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{pmatrix}$$

将字词表示为向量

- 词向量

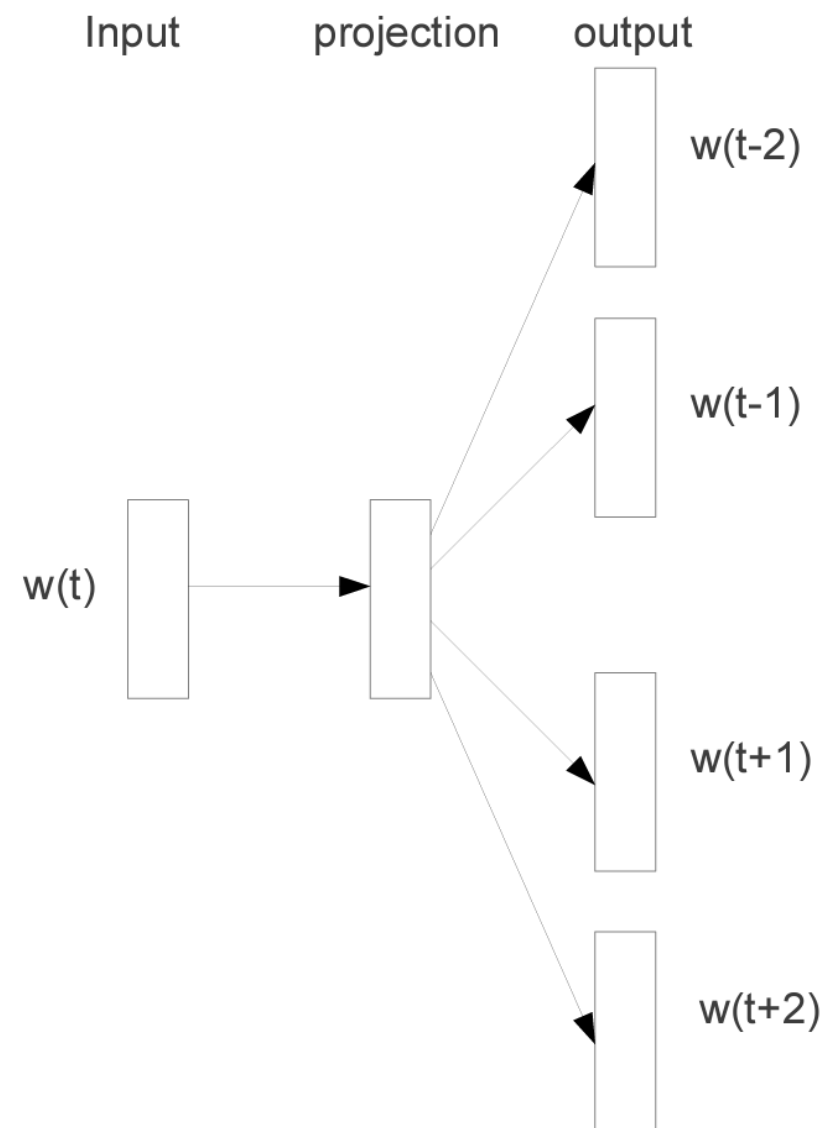
expect =

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$



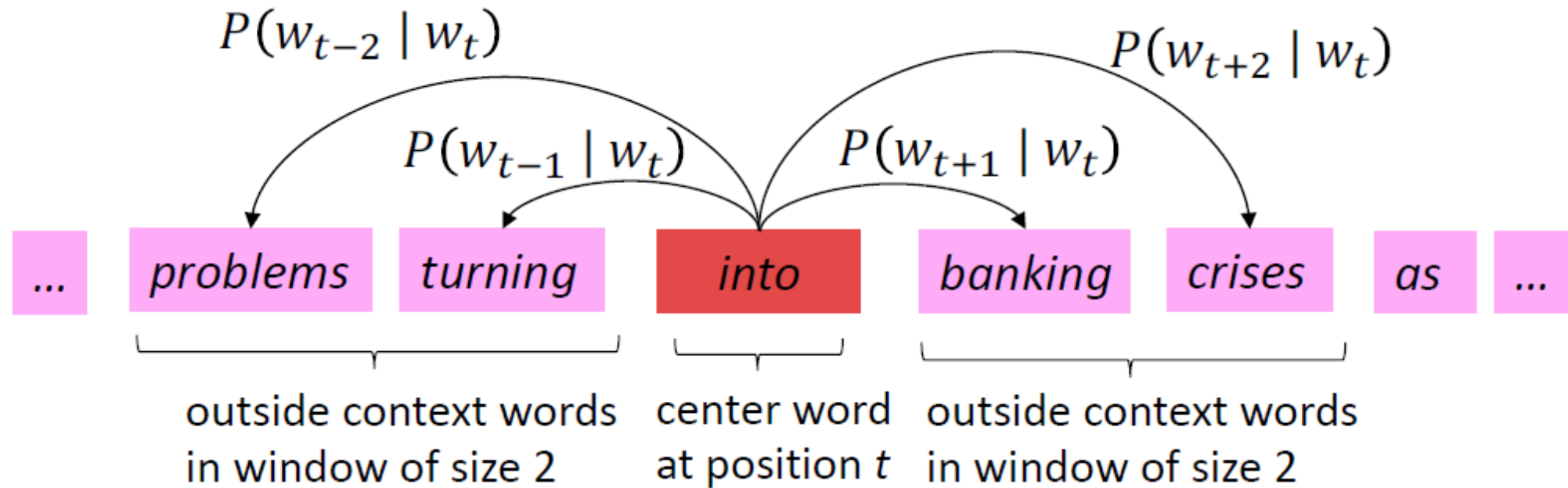
• Word2vec

- Word2vec是学习词向量表示的算法框架 (Mikolov et al., 2013) ;
- 基本思想:
 - 给定一段长文本;
 - 文本中的每个词汇表示为一个 (连续值) 向量;
 - 用一个滑动窗口 t 遍历文本, 窗口包含一个中心词 c 及其上下文词 o ;
 - 用 c 和 o 的词向量相似度**计算给定 c 的情况下 o 出现的概率** (反之亦然) ;
 - 持续调整词向量, 以最大化这个概率。



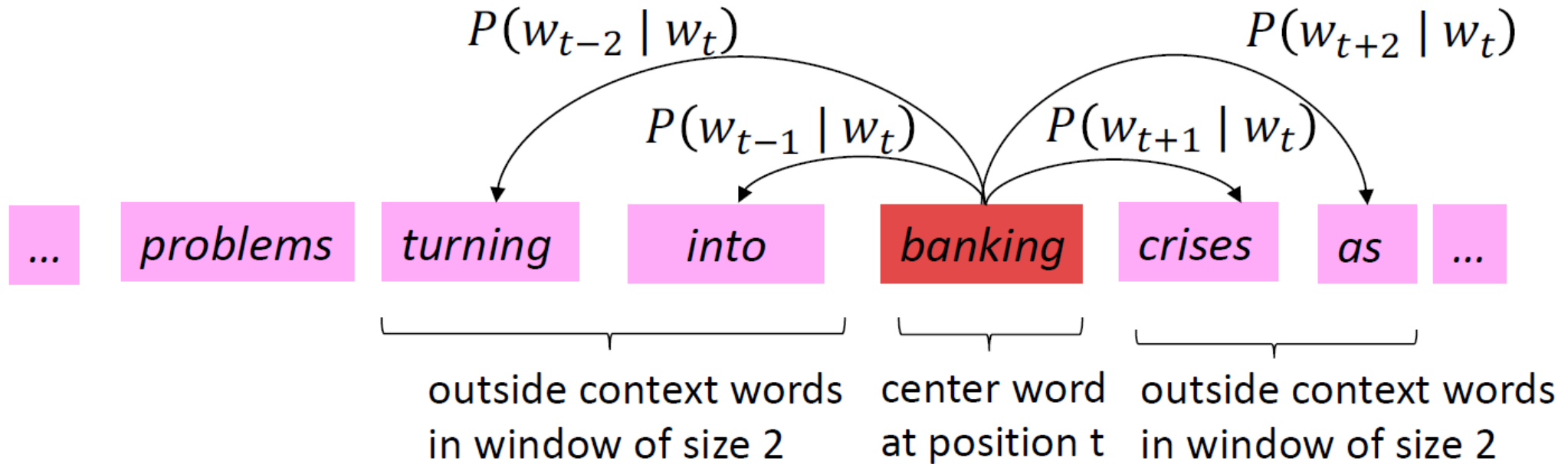
Word2vec

- Word2vec



Word2vec

- Word2vec



● 目标函数

- 对于每个位置 $t = 1, \dots, T$, 中心词为 w_t , 预测大小为 m 的窗口的上下文词出现的概率;
- 似然函数:

Likelihood = $L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$

θ is all variables to be optimized

- 优化的目标函数是**平均负对数似然**:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

- 优化目标: 最小化目标函数值 = 最大化预测准确率。

● 目标函数

- 优化的目标函数是**平均负对数似然**:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

- 如何计算 $P(w_{t+j} | w_t; \theta)$?

- 每个词 w 使用两个向量来表示:

v_w : 当 w 为中心词;

u_w : 当 w 为上下文词;

- 对于一个中心词 c 和一个上下文词 o :

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)} \quad (\text{Softmax})$$

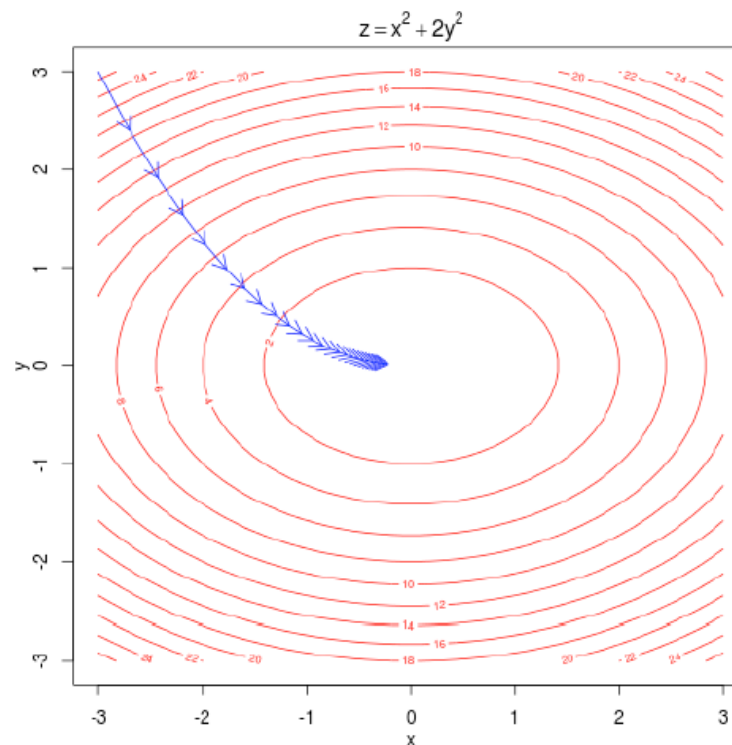
保证为正数 利用点积表示 o 和 c 的相似度

归一化

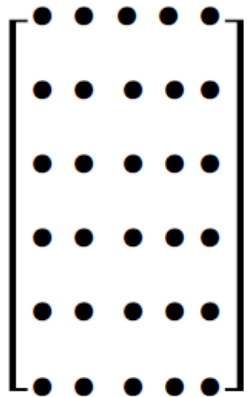
● 模型训练

- 通过梯度下降法逐步调整参数，以最小化损失函数；
- θ 表示模型的所有参数；
- 假设词向量维度为 d ，词的数量为 V ，由于每个词使用2个向量表示， θ 的长度为 $2dV$ 。

$$\theta = \begin{bmatrix} v_{aardvark} \\ v_a \\ \vdots \\ v_{zebra} \\ u_{aardvark} \\ u_a \\ \vdots \\ u_{zebra} \end{bmatrix} \in \mathbb{R}^{2dV}$$

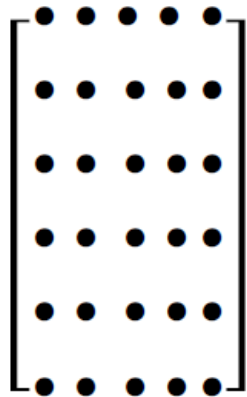


Word2vec



U

outside



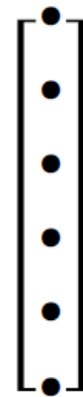
V

center



$U \cdot v_4^T$

dot product



$\text{softmax}(U \cdot v_4^T)$

probabilities

● 模型细节

■ Word2vec有两种常见的形式：

- 连续词袋模型 (Continuous bag of words, CBOW)
- 跳字模型 (Skip-gram)

■ 连续词袋模型 (CBOW) :

- 给定上下文词，预测中心词；
- 训练目标：最大化预测中心词的概率——得到表示上下文语境的词向量；

■ 跳字模型 (Skip-gram) :

- 给定中心词，预测上下文词；
- 训练目标：最大化预测上下文词的概率——得到表示词语语义的词向量。

● 模型细节

- 条件概率计算: $P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$

- 每步梯度计算都包含词典大小的（~几十万）的项的累加，计算开销很大；

- 简化计算——**负采样 (Negative sampling)**：

- 思想：训练一个**二分类模型**，用于区分真正同时出现的词和“随机”组合在一起的词（噪声）；
- 方法：取K个负样本，最大化真实共现的概率，最小化随机组合的概率；
- 修改了原来的目标函数：

$$J_{neg-sample}(\mathbf{u}_o, \mathbf{v}_c, U) = - \log \sigma(\mathbf{u}_o^T \mathbf{v}_c) - \sum_{k \in \{K \text{ sampled indices}\}} \log \sigma(-\mathbf{u}_k^T \mathbf{v}_c)$$

sigmoid rather than softmax

• 优点

- 词向量维度低，方便存储、处理，且通用性强；
- 能够很好地计算词与词之间的相似度。

```
In [10]: result = model.most_similar(u"青蛙")
```

```
In [11]: for e in result:  
         print e[0], e[1]
```

```
.....:  
老鼠 0.559612870216  
乌龟 0.489831030369  
蜥蜴 0.478990525007  
猫 0.46728849411  
鳄鱼 0.461885392666  
蟾蜍 0.448014199734  
猴子 0.436584025621  
白雪公主 0.434905380011  
蚯蚓 0.433413207531  
螃蟹 0.4314712286
```

```
In [20]: result = model.most_similar(u"清华大学")
```

```
In [21]: for e in result:  
         print e[0], e[1]
```

```
.....:  
北京大学 0.763922810555  
化学系 0.724210739136  
物理系 0.694550514221  
数学系 0.684280991554  
中山大学 0.677202701569  
复旦 0.657914161682  
师范大学 0.656435549259  
哲学系 0.654701948166  
生物系 0.654403865337  
中文系 0.653147578239
```


- **缺点**

- 词和向量是一一对应关系，无法处理一词多义的问题；
- 是一种静态的方式，无法针对特定任务做动态优化。

Word2vec

**GENSIM**
topic modelling for humans

HomeDocumentationSupportAPIAboutDonate

4.3.0

Search docs

What is Gensim?

Documentation

Core Tutorials: New Users Start Here!

Tutorials: Learning Oriented Lessons

How-to Guides: Solve a Problem

Other Resources

Core Tutorials: New Users Start Here!

Tutorials: Learning Oriented Lessons

Word2Vec Model

Doc2Vec Model

Ensemble LDA

FastText Model

Fast Similarity Queries with Annoy and Word2Vec

LDA Model

Word Mover's Distance

Please **sponsor Gensim** to help sustain this open source project!

» Documentation » Tutorials: Learning Oriented Lessons » Word2Vec Model

Note

Click [here](#) to download the full example code

Word2Vec Model

Introduces Gensim's Word2Vec model and demonstrates its use on the [Lee Evaluation Corpus](#).

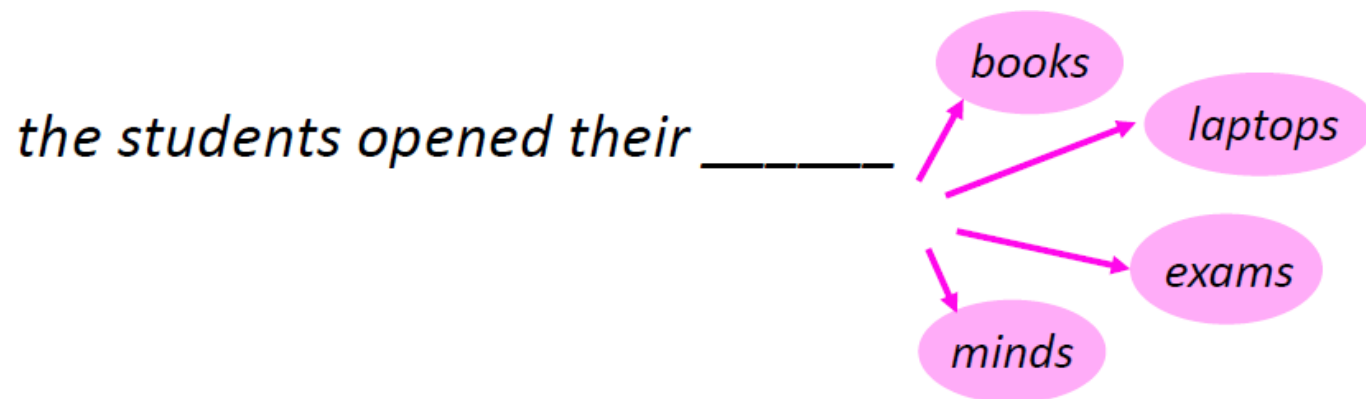
```
import logging
logging.basicConfig(format='%(asctime)s : %(levelname)s : %(message)s', level=logging.INFO)
```

In case you missed the buzz, Word2Vec is a widely used algorithm based on neural networks, commonly referred to as “deep learning” (though word2vec itself is rather shallow). Using large amounts of unannotated plain text, word2vec learns relationships between words automatically. The output are vectors, one vector per word, with remarkable linear relationships that allow us to do things like:

语言模型

- 语言模型 (Language model)

- 基本任务：预测 “下一个词”



- 给定一个单词序列 $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(t)}$, 计算下一个词 $\mathbf{x}^{(t+1)}$ 的概率分布:

$$P(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)})$$

其中, $\mathbf{x}^{(t+1)}$ 是词典 $V = \{\mathbf{w}_1, \dots, \mathbf{w}_{|V|}\}$ 中的任意单词。

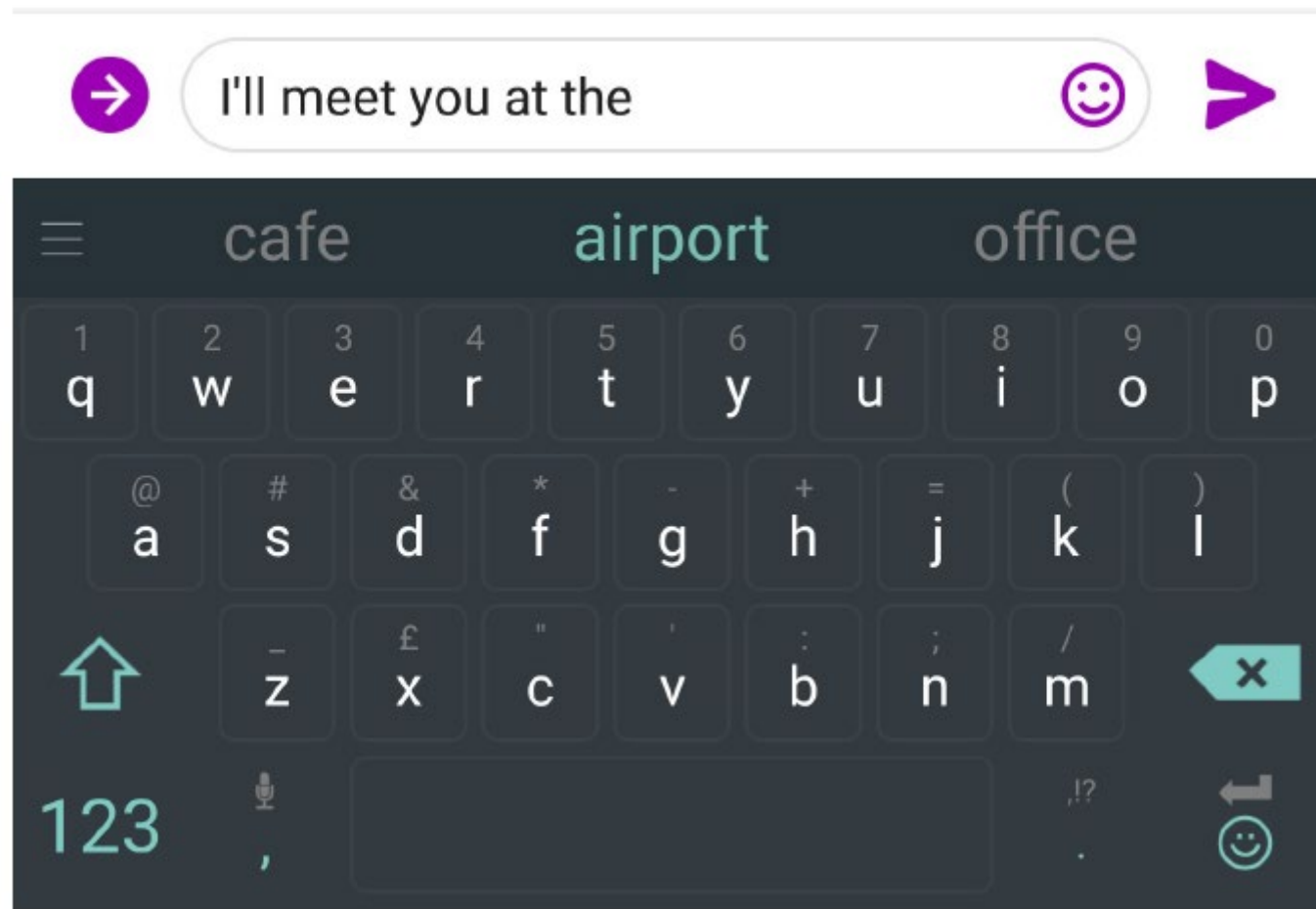
● 语言模型 (Language model)

- 从另一个视角，语言模型为某个特定语段的出现计算了概率；
- 例如，对于某个语段 $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}$ ，其出现的概率可按以下方式计算：

$$\begin{aligned} P(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}) &= P(\mathbf{x}^{(1)}) \times P(\mathbf{x}^{(2)} | \mathbf{x}^{(1)}) \times \dots \times P(\mathbf{x}^{(T)} | \mathbf{x}^{(T-1)}, \dots, \mathbf{x}^{(1)}) \\ &= \prod_{t=1}^T \underbrace{P(\mathbf{x}^{(t)} | \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(1)})} \end{aligned}$$

由语言模型计算得到

- 语言模型 (Language model)



- 语言模型 (Language model)



what is the | 

what is the **weather**
what is the **meaning of life**
what is the **dark web**
what is the **xfl**
what is the **doomsday clock**
what is the **weather today**
what is the **keto diet**
what is the **american dream**
what is the **speed of light**
what is the **bill of rights**

[Google Search](#) [I'm Feeling Lucky](#)

● n-gram语言模型

- 在计算语言学中，n-gram是指文本中连续的n个item（例如n个词）；

the students opened their _____

- **uni**grams: “the”, “students”, “opened”, “their”
 - **bi**grams: “the students”, “students opened”, “opened their”
 - **tri**grams: “the students opened”, “students opened their”
 - **four**-grams: “the students opened their”
- n-gram语言模型：计算不同n-gram出现的频率，用于预测 “下一个词”。

• n-gram语言模型

- 马尔科夫假设: $\mathbf{x}^{(t+1)}$ 仅依赖其前面出现的 $n-1$ 个词;

$$\begin{aligned} P(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)}) &= P(\mathbf{x}^{(t+1)} | \overbrace{\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}}^{n-1 \text{ words}}) \\ &= \frac{P(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})}{P(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})} \end{aligned}$$

n-gram
(n-1)-gram

- 根据大数定律, 我们统计频数以得到概率的近似:

$$\frac{P(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})}{P(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})} \approx \frac{\text{count}(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})}{\text{count}(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})}$$

• n-gram语言模型

- 案例：假如我们试图建立一个4-gram语言模型

as the proctor started the clock, the students opened their _____

舍弃

仅考虑前面3个词

$$P(\boldsymbol{w} | \text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

- 假设语料库中，students opened their 共出现了1000次；
- students opened their books出现了400次： $P(\text{books} | \text{students opened their}) = 0.4$
- students opened their exams出现了100次： $P(\text{exams} | \text{students opened their}) = 0.1$

- 利用n-gram语言模型进行文本生成

Today the _____

得到概率分布，即前2个词是 today the 的条件下，接下来出现某个词的概率



company	0.153
bank	0.153
price	0.077
italian	0.039
emirate	0.039
...	

例如， price出现的概率为0.077

- 利用n-gram语言模型进行文本生成

Today the price _____

得到概率分布，即前2个词是 the price 的条件下，接下来出现某个词的概率



of	0.308
for	0.050
it	0.046
to	0.046
is	0.031
...	

例如，**of** 出现的概率为 **0.308**

- 利用n-gram语言模型进行文本生成

Today the price of _____

得到概率分布，即前2个词是 price of 的条件下，接下来出现某个词的概率



the	0.072
18	0.043
oil	0.043
its	0.036
gold	0.018
...	

例如，**gold** 出现的概率为 **0.018**

- 利用n-gram语言模型进行文本生成

today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .

- n-gram语言模型的不足

the students opened their _____

问题1: students opened their w 可能从未在语料库出现过

$$P(w|\text{students opened their}) = \frac{\text{count}(\text{students opened their } w)}{\text{count}(\text{students opened their})}$$

问题2: students opened their 也可能从未在语料库出现过

- 稀疏性问题;
- 对于n-gram模型, 随着n的增大, 模型的稀疏性也随之增长。

- 问题：如何用神经网络构建语言模型？

固定窗口神经语言模型

输出分布

$$\hat{y} = \text{softmax}(Uh + b_2) \in \mathbb{R}^{|V|}$$

Hidden layer

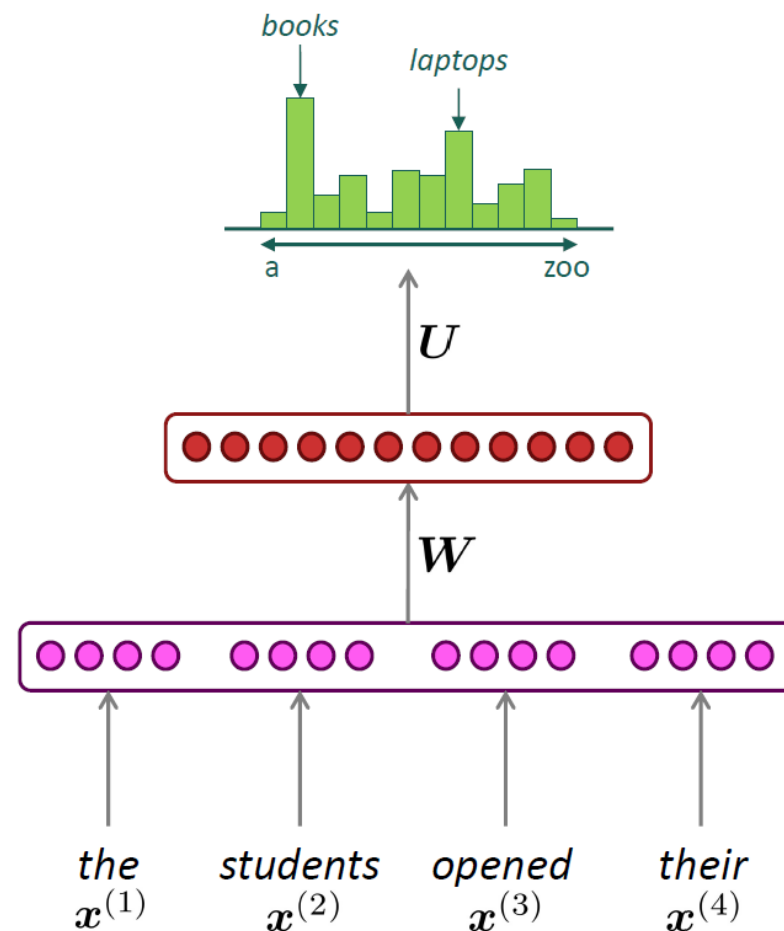
$$h = f(We + b_1)$$

词嵌入

$$e = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

单词

$$x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}$$



● 问题：如何用神经网络构建语言模型？

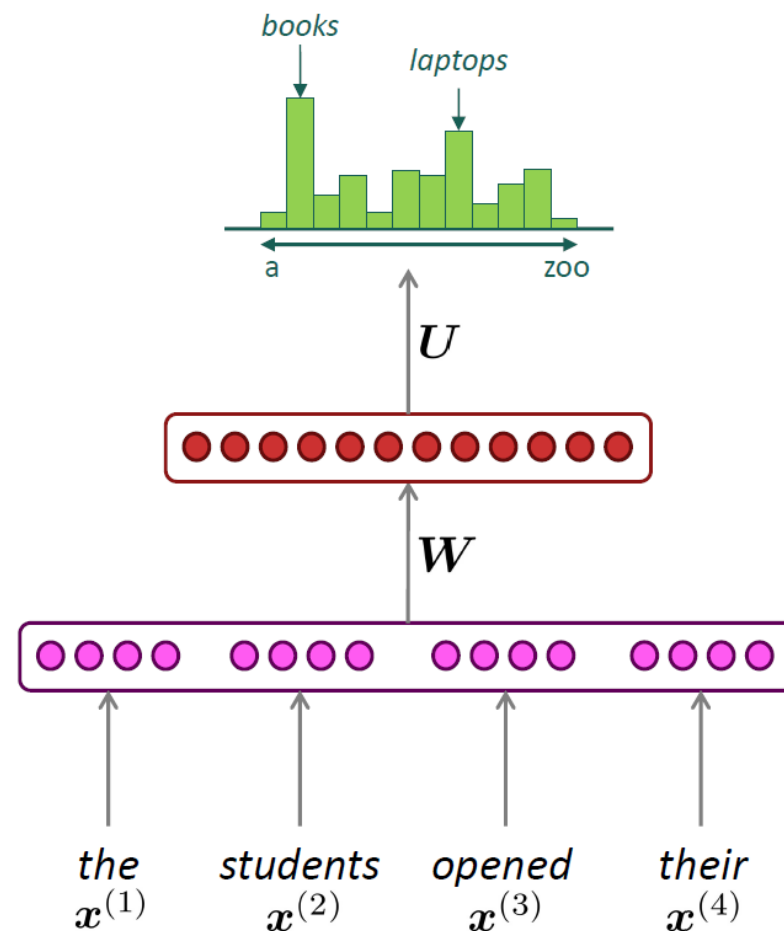
固定窗口神经语言模型

优点：

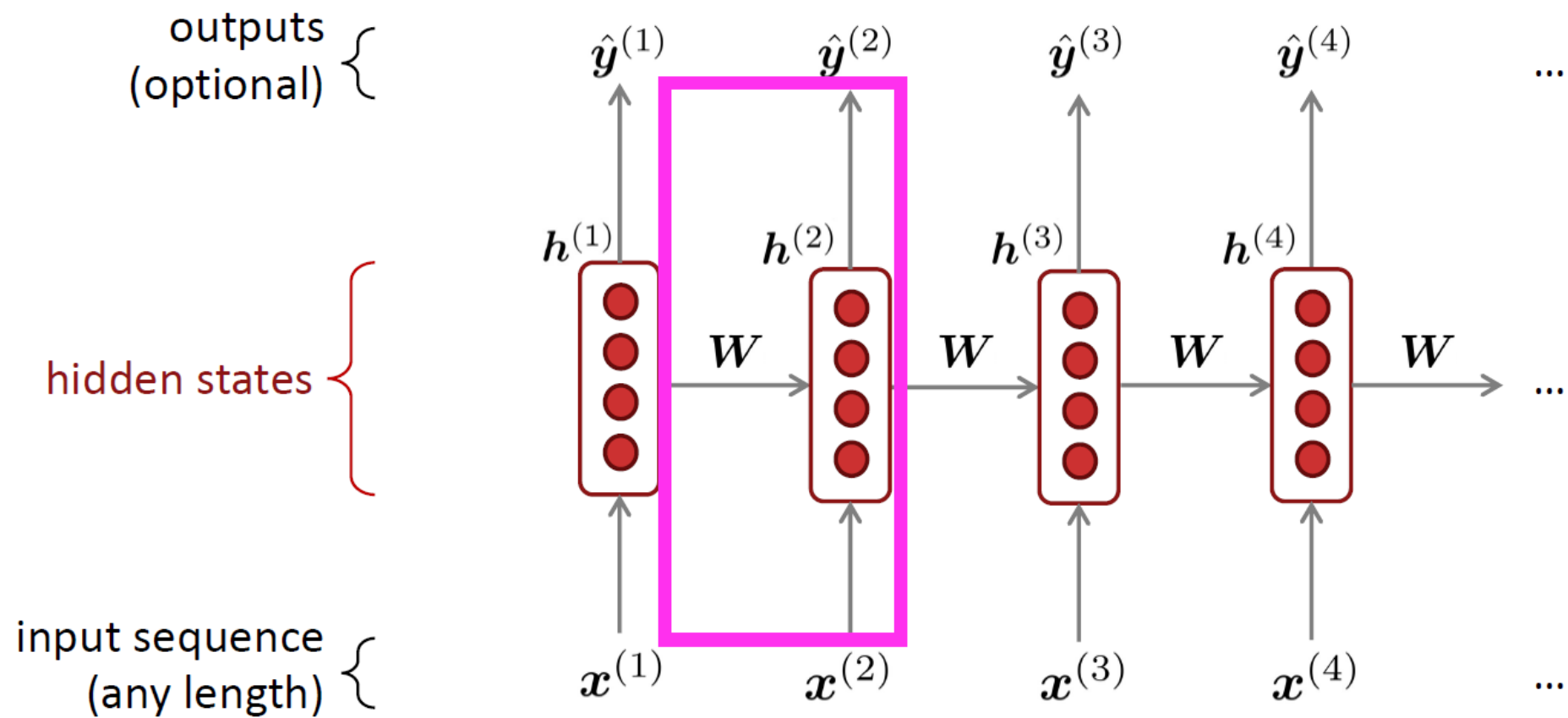
- 与n-gram语言模型相比，解决了稀疏问题；

不足：

- 需要固定窗口大小，如果增大窗口，将导致权重矩阵 W 规模增大；
- 不存在对称性， $x^{(1)}$ 、 $x^{(2)}$ 交换位置将导致完全不同的结果。



- 基于RNN构建语言模型



● 基于RNN构建语言模型

输出分布

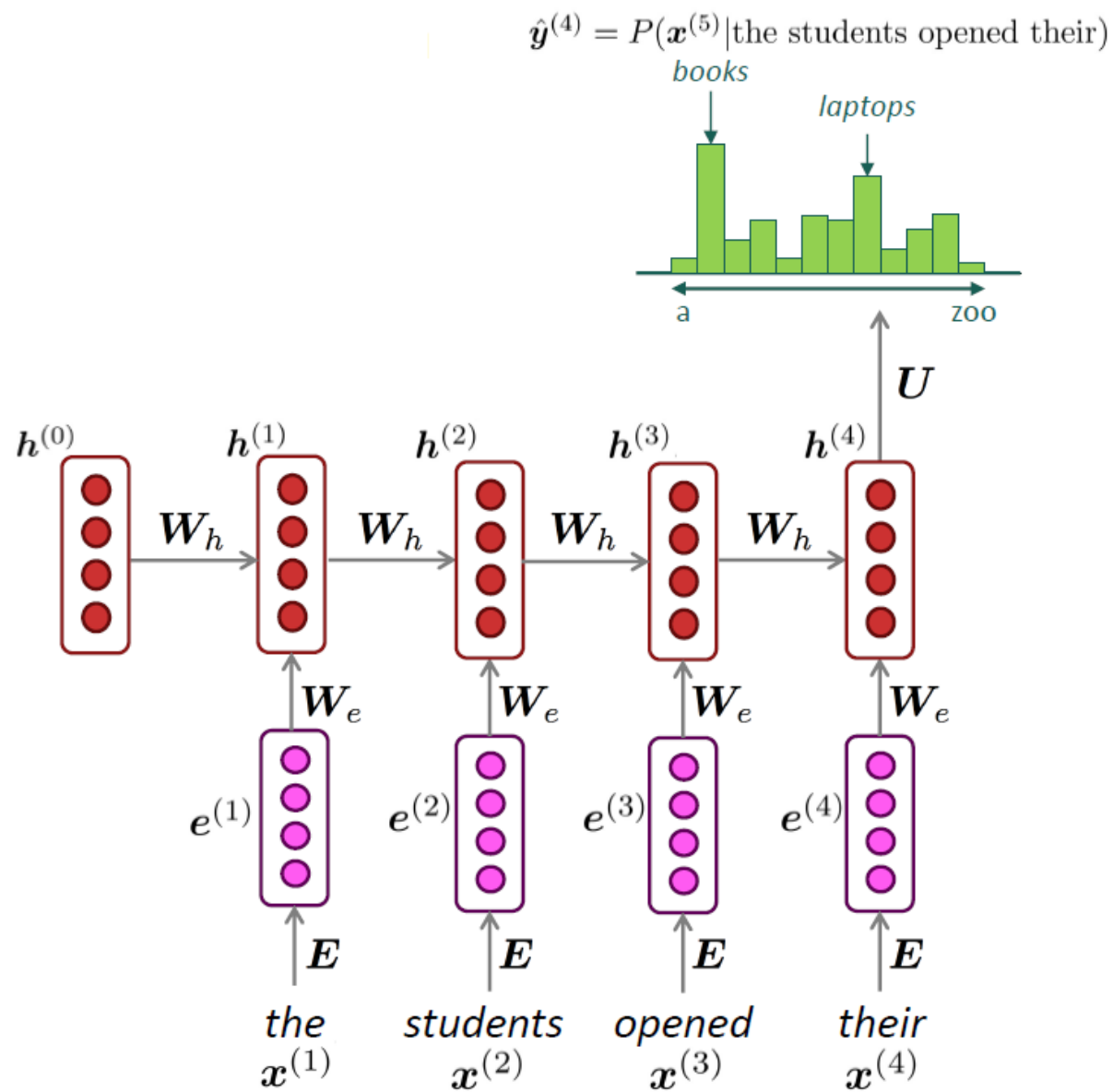
$$\hat{y}^{(t)} = \text{softmax} \left(U h^{(t)} + b_2 \right) \in \mathbb{R}^{|V|}$$

Hidden layer

$$h^{(t)} = \sigma \left(W_h h^{(t-1)} + W_e e^{(t)} + b_1 \right)$$

词嵌入 $e^{(t)} = E x^{(t)}$

单词 $x^{(t)} \in \mathbb{R}^{|V|}$



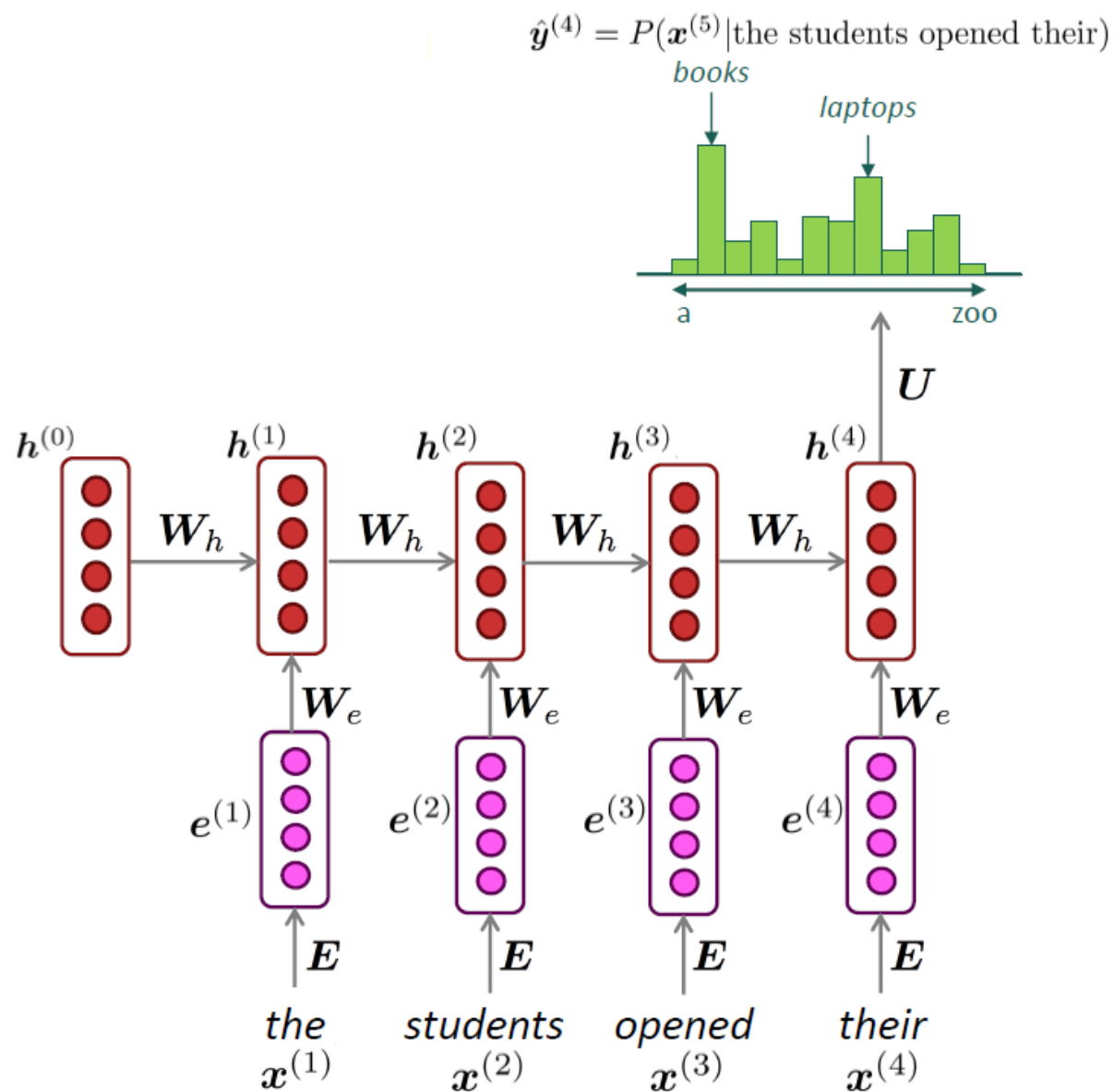
● 基于RNN构建语言模型

优点:

- 文本长度不受限制;
- 随着文本长度的增长, 由于共享权重的机制, 模型大小不会增长

不足:

- RNN存在长程依赖问题;
- 训练、推理较慢。



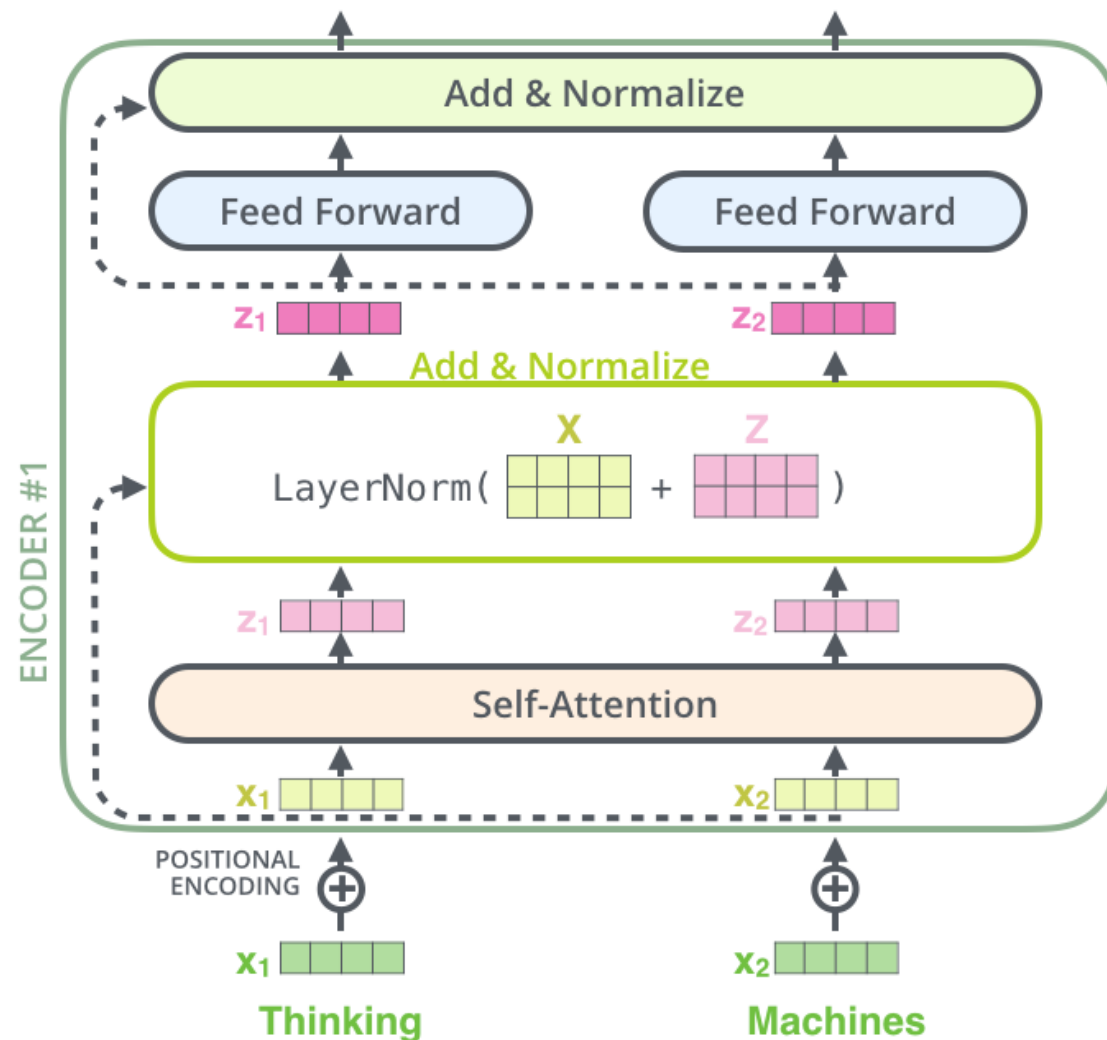
● 基于Transformer构建语言模型

优点：

- 文本长度不受限制；
- 比RNN更好地上下文关联能力；

不足：

- 训练、推理较慢，消耗计算资源多。



Seq2Seq与机器翻译

- 机器翻译 (Machine Translation, MT)

- 任务：将源语言 (Source language) 语段 x 翻译为目标语言 (Target language) 语段 y ;

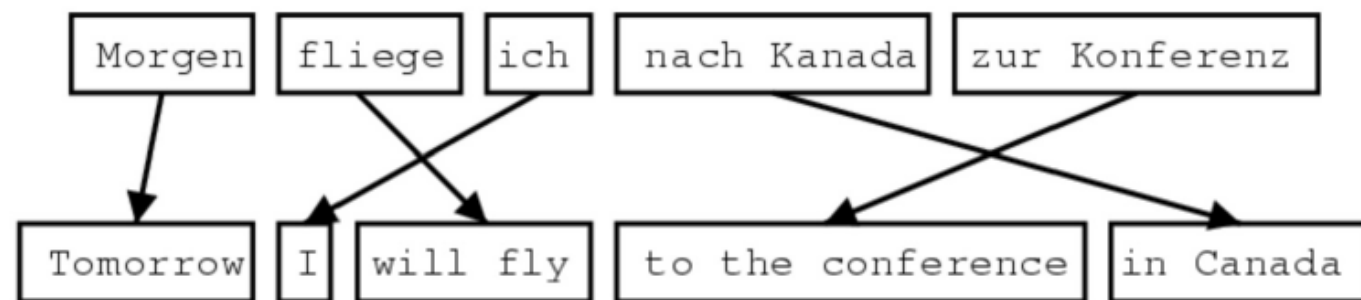
x : *L'homme est né libre, et partout il est dans les fers*



y : *Man is born free, but everywhere he is in chains*

● 早期的机器翻译

- 机器翻译相关研究最早出现在20世纪50年代；
- 军事、情报部门投入大量资源进行机器翻译的研究；
- 主要是进行不同语言之间的单词替换，效果很差。



1519年600名西班牙人在墨西哥登陆，去征服**几百万人口**
的阿兹特克帝国，初次交锋他们损兵三分之二。

In 1519, six hundred Spaniards landed in Mexico to conquer **the Aztec Empire** **with a population of a few million**. They lost two thirds of their soldiers in the first clash.

translate.google.com (2009): 1519 600 Spaniards landed in Mexico, **millions of people** to **conquer the Aztec empire**, the first two-thirds of soldiers against their loss.

translate.google.com (2013): 1519 600 Spaniards landed in Mexico **to conquer the Aztec empire**, **hundreds of millions of people**, the initial confrontation loss of soldiers two-thirds.

translate.google.com (2015): 1519 600 Spaniards landed in Mexico, **millions of people** to **conquer the Aztec empire**, the first two-thirds of the loss of soldiers they clash.

- 统计机器翻译 (1990s~2010s)

- 源语言句子 x , 目标语言句子 y ;
- 统计机器翻译建立了以下目标:

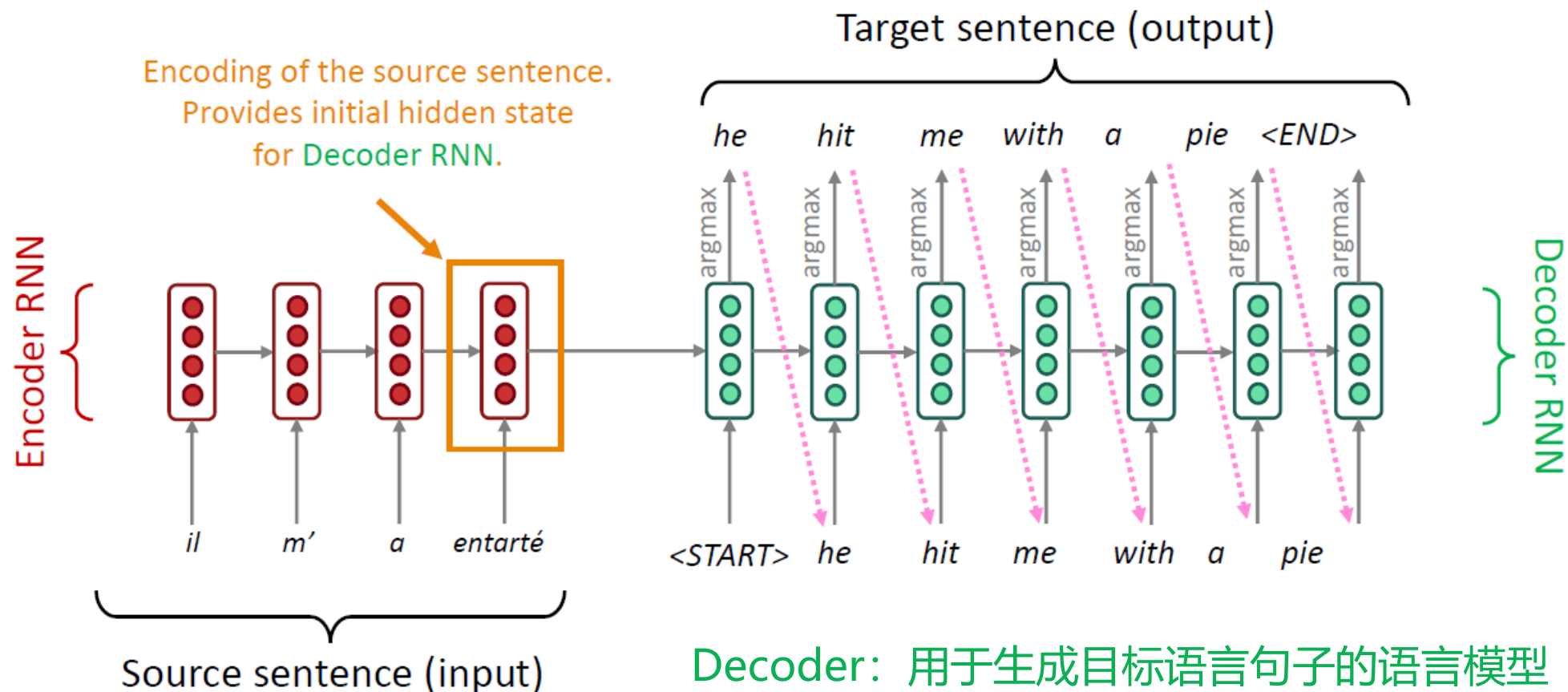
$$\operatorname{argmax}_y P(y|x)$$

- 根据贝叶斯定理, 上式可以改写为:

$$\operatorname{argmax}_y P(x|y)P(y)$$

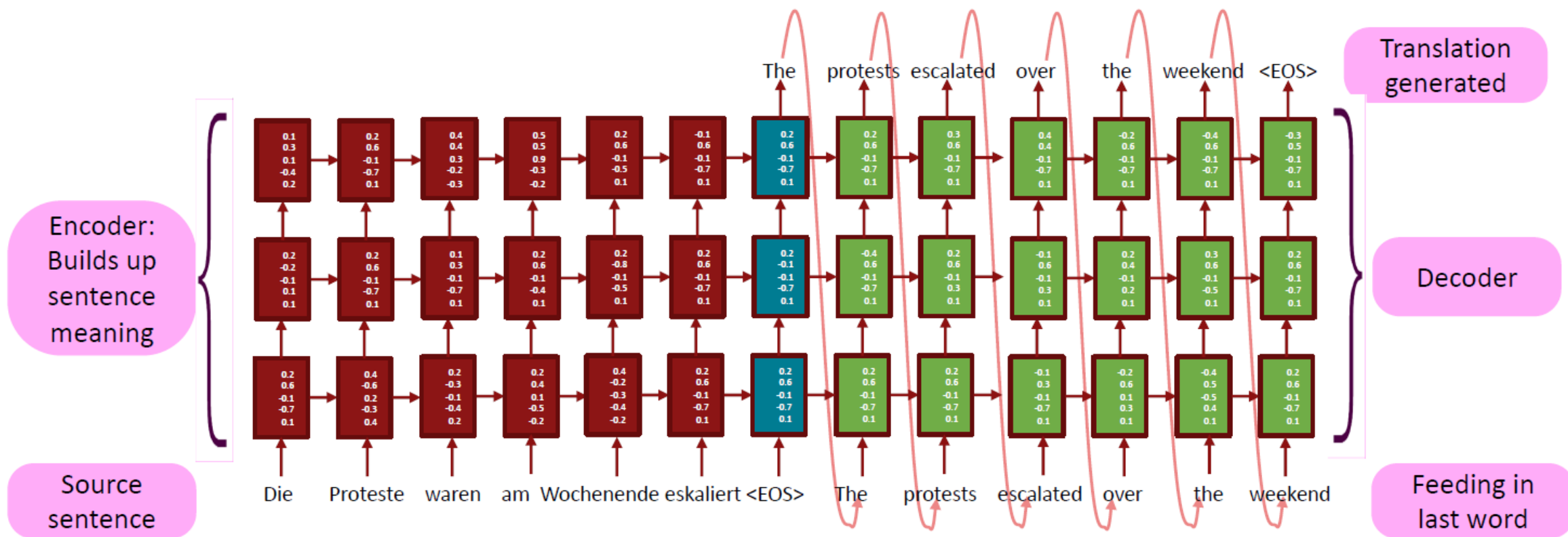
- $P(x|y)$: 翻译模型——通过多语言平行数据学习;
- $P(y)$: 语言模型——通过某种特定语言的数据进行学习。

● 基于神经网络的机器翻译——Seq2seq模型



Encoder: 将源语言句子编码为隐层特征

● 基于神经网络的机器翻译——Seq2seq模型



大规模语言模型

大规模语言模型

● 产生背景：

- 计算机视觉领域采用ImageNet数据集对模型进行预训练，使得模型可以通过海量图像充分学习如何提取特征，然后再根据任务目标进行模型精调；
- 受此影响，自然语言处理领域基于预训练语言模型的方法也逐渐成为主流；
- 以GPT为代表的基于Transformer的大规模预训练语言模型的出现，使得自然语言处理全面进入了**预训练微调范式**新时代。

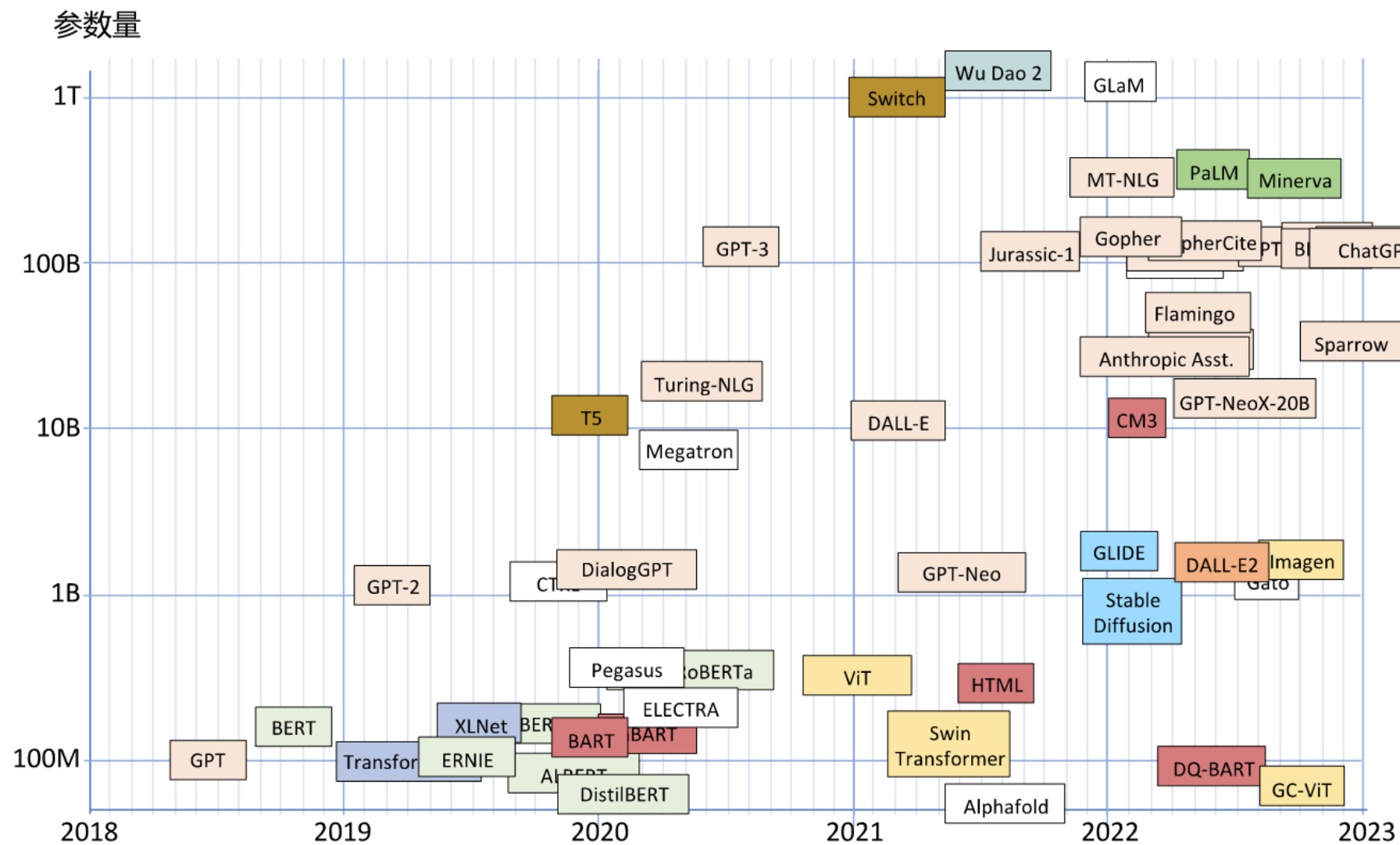
● 优势：

- 将预训练模型应用于下游任务时，不需要了解太多的任务细节，不需要设计特定的神经网络结构，只需要**“微调” (finetune)** 预训练模型，即使用具体任务的标注数据在预训练语言模型上进行监督训练，就可以取得显著的性能提升。

- “百模大战”：
 - 2020年，Open AI发布了包含1750亿参数的生成式大规模预训练语言模型GPT 3（Generative Pre-trained Transformer 3）；
 - 此后，Google、Meta、百度等公司和研究机构都纷纷发布了PaLM、LaMDA、T0等不同大规模语言模型（Large Language Model，LLM），也称大模型。

模型名称	参数量	训练单词数	研发机构	发布时间
ChatGPT	1750 亿	3000 亿	OpenAI	2022 年 11 月
Galactica	1200 亿	4500 亿	Meta AI	2022 年 11 月
BLOOMZ	1760 亿	3660 亿	BigScience	2022 年 11 月
U-PaLM	5400 亿	7800 亿	Google Research	2022 年 10 月
CodeGeeX	130 亿	8500 亿	清华大学	2022 年 9 月
PaLM	5400 亿	7800 亿	Google Research	2022 年 4 月
ERNIE 3.0 Titan	2600 亿	–	Baidu	2021 年 12 月
FLAN	1370 亿	–	Google	2021 年 9 月
GPT-3	1750 亿	3000 亿	OpenAI	2020 年 5 月
T5	110 亿	340 亿	Google	2019 年 10 月
RoBERTa	3.55 亿	22000 亿	Meta AI	2019 年 7 月
GPT-2	15 亿	100 亿	OpenAI	2019 年 2 月
BERT	3 亿	1370 亿	Google	2018 年 10 月
GPT-1	1 亿	–	OpenAI	2018 年 6 月

大规模语言模型



- 整体过程可以分为三个阶段：

- 第一个阶段是**基础大模型训练**，该阶段主要完成长距离语言模型的预训练，通过代码预训练使得模型具备代码生成的能力；
- 第二阶段是**指令微调（Instruct Tuning）**，通过给定指令进行微调的方式使得模型具备完成各类任务的能力；
- 第三个阶段是**类人对齐**，加入更多人工提示词，并利用有监督微调并结合基于强化学习的方式，使得模型输出更贴合人类需求。

● 基础大模型训练：

- 训练数据集主要包含经过过滤的Common Crawl数据集、WebText2、Books1、Books2以及英文Wikipedia等数据集；
- 其中Common Crawl的原始数据有45TB，进行过滤后保留了570GB的数据；
- 通过子词方式对语料进行切分，大约一共包含5000亿子词；
- 为了保证模型使用更多高质量数据进行训练，在GPT-3训练时，根据语料来源的不同，设置不同的采样权重，在完成3000亿子词训练时，英文Wikipedia的语料平均训练轮数为3.4次，而Common Crawl和Books 2为0.44次和0.43次；
- 据估计，OpenAI使用了约1万块NVIDIA A100 GPU来训练GPT-3.5模型，后来为了进一步满足服务器需求，将GPU数量提升到了3万块以上。

[The Data ▼](#)[Resources ▼](#)[Community ▼](#)[About ▼](#)[Search ▼](#)[Contact Us](#)

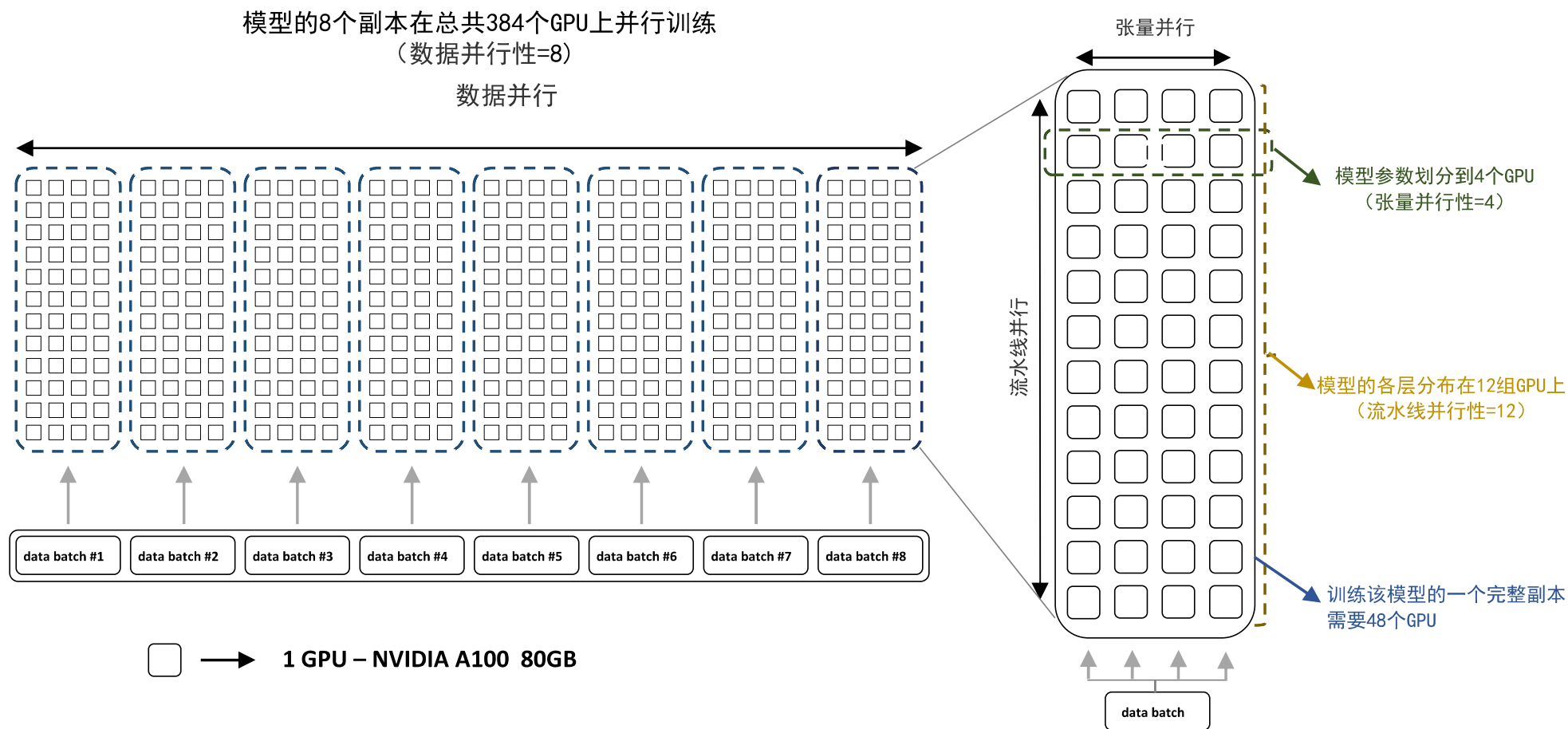
Common Crawl
maintains a **free, open
repository** of web crawl
data that can be used by
anyone.



大模型训练

● 基础大模型训练：

- BLOOM（与GPT3类似的模型）使用Megatron-DeepSpeed分布式训练框架



● 指令微调：

- 预训练语言模型可以根据任务数据进行微调（Fine-tuning），这种范式能够应用于中等规模参数量（几百万到几亿规模）的预训练模型，但是针对成百上千亿参数规模的大模型，针对每个任务都进行微调的计算开销和时间成本几乎都是不可接受的；
- 因此，研究人员们提出了指令微调（Instruction Finetuning）方案，将大量各类型任务，统一为生成式自然语言理解框架，并构造训练语料进行微调。

例如，可以将情感倾向分析任务，通过如下指令，将贬义和褒义的分类问题转换到生成式自然语言理解框架：

For each snippet of text, label the sentiment of the text as positive or negative.

Text: this film seems thirsty for reflection, itself taking on adolescent qualities.

Label: [positive / negative]

大模型训练

训练目标不包含思维链

指令
不包含样例

Answer the following
yes/no question.

→ yes

Can you write a whole
Haiku in a single tweet?

指令
包含样例

Q: Answer the following
yes/no question.
Could a dandelion suffer
from hepatitis?

A: no

→ yes

Q: Answer the following
yes/no question.
Can you write a whole Haiku
in a single tweet?
A:

训练目标包含思维链

Answer the following yes/no question
by reasoning step-by-step.

→

Can you write a whole Haiku in a
single tweet?

A haiku is a japanese
three-line poem. That
is short enough to fit in
280 characters. The
answer is yes.

Q: Answer the following yes/no question by
reasoning step-by-step.
Could a dandelion suffer from hepatitis?
A: Hepatitis only affects organisms with livers.
Dandelions don't have a liver. The answer is no.

→

Q: Answer the following yes/no question by
reasoning step-by-step.
Can you write a whole Haiku in a single tweet?
A:

A haiku is a japanese
three-line poem. That
is short enough to fit in
280 characters. The
answer is yes.

● 类人对齐：

- 经过指令微调后的模型，虽然在开放领域任务能力表现优异，但是模型输出的结果通常是简单答案，与人类的回答相差很大，因此需要进一步优化模型，使其可以生成更加贴近人类习惯的文本内容；
- 自然语言处理语料集合中所包含的答案，通常都是简短的标签，基本没有类似人类回答的有监督数据，如果将自然语言处理任务数据集进行人工改造，所需要的时间成本和人工成本过于高昂；
- Open AI提出了使用**基于人类反馈的强化学习方法（Reinforcement Learning from Human Feedback, RLHF）**，从而大幅度降低了数据集构建成本，并达到了非常好的效果。

● 类人对齐：

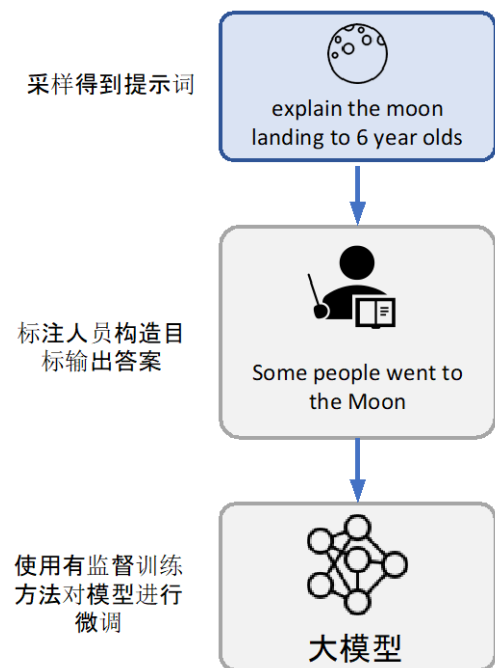
■ RLHF算法主要分为以下三个步骤：

- **Step1. 有监督微调**：人类标注者准备一组提示和期望结果，对预训练模型进行有监督微调；
- **Step2. 创建奖励模型**：人类标注者利用模型生成一组对比数据，也就是几对指令、答案，并根据偏好对它们进行排序，根据这些排序数据训练奖励模型，以便在没有人为干预的情况下自动对强化学习智能体的输出进行排序；
- **Step3. 使用奖励模型训练 RL 策略**：创建一个反馈环来训练和微调强化学习策略，利用奖励模型生成最优回答。这个过程会迭代进行，直到达到预期的性能水平。

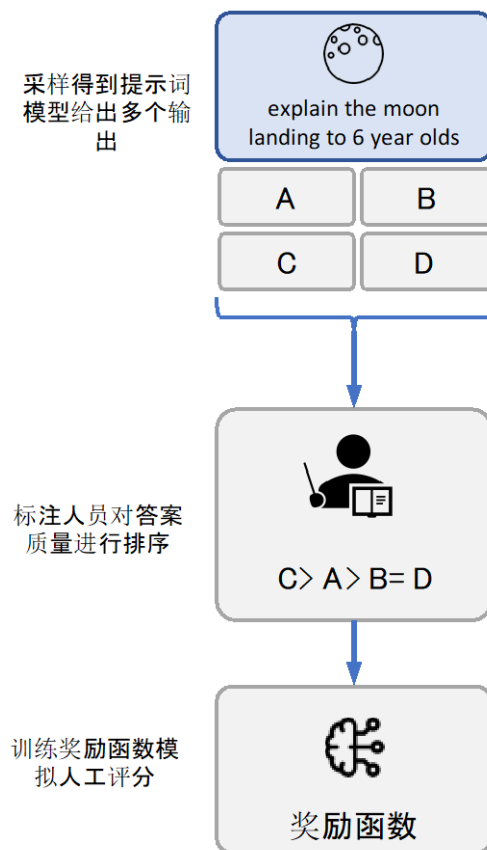
大模型训练

● 类人对齐:

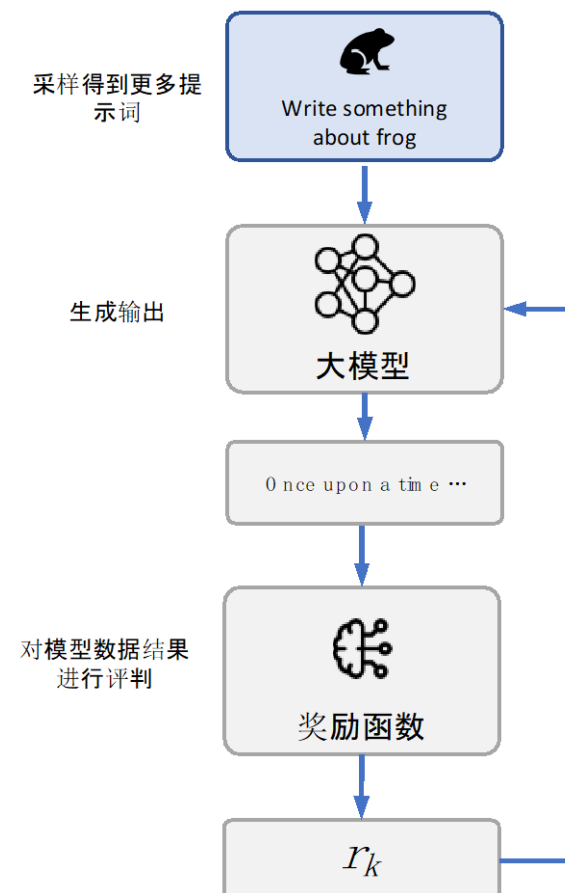
步骤1：收集初始数据
初步训练大模型



步骤2：收集打分数据
并训练奖励函数



步骤3：利用强化学习
机制根据奖励得分进一步
优化大模型



更多学习资源

- **Stanford CS224n (by Chris Manning) :**
 - <https://web.stanford.edu/class/cs224n/>
- **Coursera 利用分类和向量空间进行自然语言处理:**
 - <https://www.coursera.org/learn/classification-vector-spaces-in-nlp>

谢谢!